

## Chương 15 – Xử lý chuỗi bằng RNN và CNN

*Notebook này chứa tất cả các mã nguồn mẫu và lời giải cho các bài tập trong chương 15.*

 [Open in Colab](#)  [Open in Kaggle](#)

### ▼ Cài đặt (Setup)

Dự án này yêu cầu Python 3.7 trở lên:

```
import sys

assert sys.version_info >= (3, 7)
```

Và TensorFlow  $\geq 2.8$ :

```
from packaging import version
import tensorflow as tf

assert version.parse(tf.__version__) >= version.parse("2.8.0")
```

Như chúng ta đã làm ở các chương trước, hãy định nghĩa kích thước phông chữ mặc định để làm cho các hình ảnh đẹp hơn:

```
import matplotlib.pyplot as plt

plt.rc('font', size=14)
plt.rc('axes', labelsz=14, titlesz=14)
plt.rc('legend', fontsize=14)
plt.rc('xtick', labelsz=10)
plt.rc('ytick', labelsz=10)
```

Và hãy tạo thư mục `images/rnn` (nếu nó chưa tồn tại), và định nghĩa hàm `save_fig()` được sử dụng trong notebook này để lưu các hình ảnh ở độ phân giải cao cho sách:

```
from pathlib import Path

IMAGES_PATH = Path() / "images" / "rnn"
IMAGES_PATH.mkdir(parents=True, exist_ok=True)

def save_fig(fig_id, tight_layout=True, fig_extension="png", resolution=300):
    path = IMAGES_PATH / f"{fig_id}.{fig_extension}"
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
```

Chương này có thể chạy rất chậm nếu không có GPU, vì vậy hãy kiểm tra xem có GPU hay không, nếu không sẽ đưa ra cảnh báo:

```
if not tf.config.list_physical_devices('GPU'):
    print("No GPU was detected. Neural nets can be very slow without a GPU.")
    if "google.colab" in sys.modules:
        print("Go to Runtime > Change runtime and select a GPU hardware "
              "accelerator.")
    if "kaggle_secrets" in sys.modules:
        print("Go to Settings > Accelerator and select GPU.")
```

### ▼ RNN cơ bản (Basic RNNs)

Hãy tải dữ liệu lượt hành khách (ridership) từ dự án [ageron/data](#). Dữ liệu này ban đầu đến từ Chicago's Transit Authority, và được tải xuống từ [Chicago's Data Portal](#).

**Cảnh báo:** trong các phiên bản Keras gần đây, `get_file()` hiện đóng gói thư mục đã giải nén bên trong một thư mục có tên kết thúc bằng `_extracted`, vì vậy đoạn mã sau đây sẽ kiểm tra điều đó:

```
filepath = tf.keras.utils.get_file(
    "ridership.tgz",
    "https://github.com/ageron/data/raw/main/ridership.tgz",
    cache_dir=".",
    extract=True
)
if "_extracted" in filepath:
    ridership_path = Path(filepath) / "ridership"
else:
    ridership_path = Path(filepath).with_name("ridership")
```

Downloading data from <https://github.com/ageron/data/raw/main/ridership.tgz>  
108512/108512 [=====] - 0s 0us/step

```
import pandas as pd
from pathlib import Path

path = Path("datasets/ridership/CTA_-_Ridership_-_Daily_Boarding_Totals.csv")
df = pd.read_csv(path, parse_dates=["service_date"])
df.columns = ["date", "day_type", "bus", "rail", "total"] # shorter names
df = df.sort_values("date").set_index("date")
df = df.drop("total", axis=1) # no need for total, it's just bus + rail
df = df.drop_duplicates() # remove duplicated months (2011-10 and 2014-07)
```

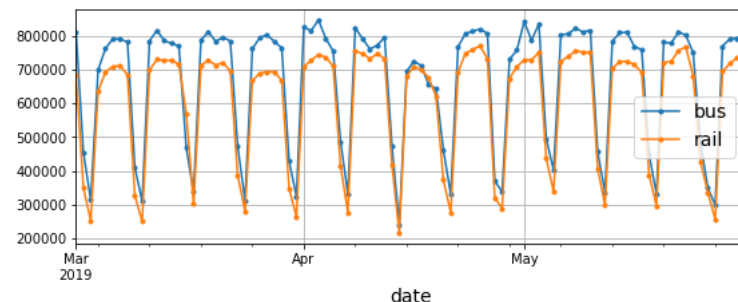
`df.head()`

|            | day_type | bus    | rail   |
|------------|----------|--------|--------|
| date       |          |        |        |
| 2001-01-01 | U        | 297192 | 126455 |
| 2001-01-02 | W        | 780827 | 501952 |
| 2001-01-03 | W        | 824923 | 536432 |
| 2001-01-04 | W        | 870021 | 550011 |
| 2001-01-05 | W        | 890426 | 557917 |

Hãy xem xét vài tháng đầu năm 2019 (lưu ý rằng Pandas coi các ranh giới phạm vi là bao gồm cả hai đầu):

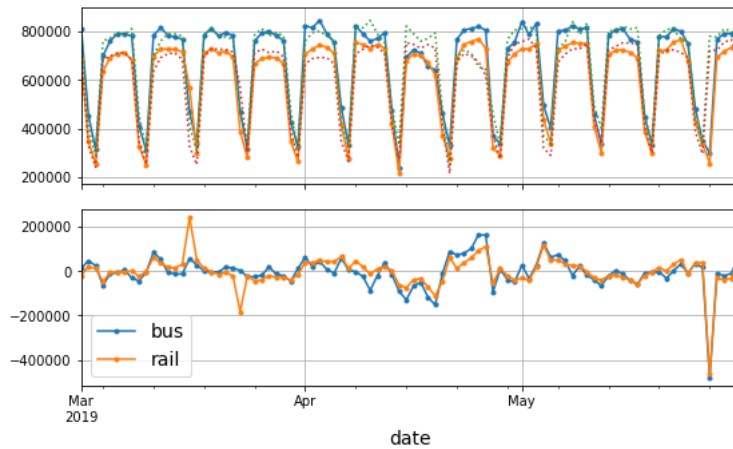
```
import matplotlib.pyplot as plt

df["2019-03":"2019-05"].plot(grid=True, marker=".", figsize=(8, 3.5))
save_fig("daily_ridership_plot") # extra code - saves the figure for the book
plt.show()
```



```
diff_7 = df[["bus", "rail"]].diff(7)["2019-03":"2019-05"]

fig, axs = plt.subplots(2, 1, sharex=True, figsize=(8, 5))
df.plot(ax=axs[0], legend=False, marker=".") # original time series
df.shift(7).plot(ax=axs[0], grid=True, legend=False, linestyle=":") # lagged
diff_7.plot(ax=axs[1], grid=True, marker=".") # 7-day difference time series
axs[0].set_ylim([170_000, 900_000]) # extra code - beautifies the plot
save_fig("differencing_plot") # extra code - saves the figure for the book
plt.show()
```



```
list(df.loc["2019-05-25":"2019-05-27"]["day_type"])
```

```
['A', 'U', 'U']
```

Sai số tuyệt đối trung bình (MAE), còn được gọi là độ lệch tuyệt đối trung bình (MAD):

```
diff_7.abs().mean()
```

```
bus    43915.608696
rail   42143.271739
dtype: float64
```

Sai số phần trăm tuyệt đối trung bình (MAPE):

```
targets = df[["bus", "rail"]]["2019-03":"2019-05"]
(diff_7 / targets).abs().mean()
```

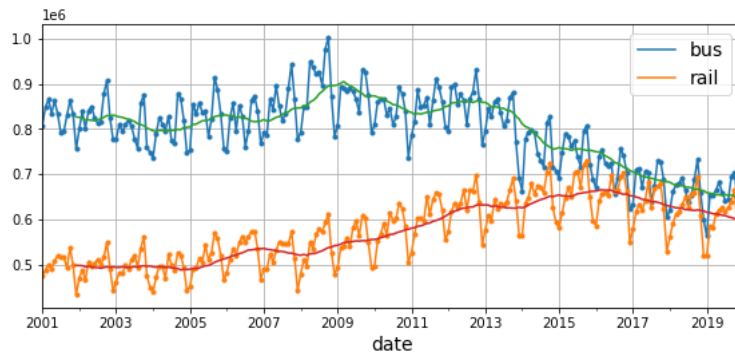
```
bus    0.082938
rail   0.089948
dtype: float64
```

Bây giờ hãy nhìn vào tính thời vụ hàng năm và các xu hướng dài hạn:

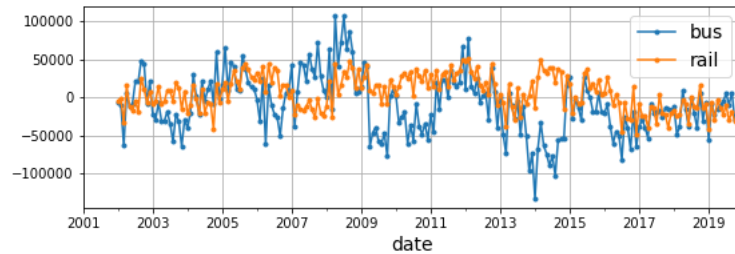
**Cảnh báo:** API của Pandas để tính giá trị trung bình cho mỗi tháng đã thay đổi khá nhiều, vì vậy mã bên dưới sẽ thử API mới nhất, nếu không được sẽ quay lại API cũ hơn, và nếu vẫn thất bại, nó sẽ sử dụng mã trong sách.

```
period = slice("2001", "2019")
try:
    df_monthly = df.select_dtypes(include="number").resample('ME').mean() # compute the mean for each month
    rolling_average_12_months = df_monthly.loc[period].rolling(window=12).mean()
except ValueError as ex:
    try:
        df_monthly = df.select_dtypes(include="number").resample('M').mean() # compute the mean for each month
        rolling_average_12_months = df_monthly.loc[period].rolling(window=12).mean()
    except ValueError as ex:
        df_monthly = df.resample('M').mean() # compute the mean for each month
        rolling_average_12_months = df_monthly[period].rolling(window=12).mean()

fig, ax = plt.subplots(figsize=(8, 4))
df_monthly[period].plot(ax=ax, marker=".")
rolling_average_12_months.plot(ax=ax, grid=True, legend=False)
save_fig("long_term_ridership_plot") # extra code - saves the figure for the book
plt.show()
```



```
df_monthly.diff(12)[period].plot(grid=True, marker=".", figsize=(8, 3))
save_fig("yearly_diff_plot") # extra code - saves the figure for the book
plt.show()
```



Nếu chạy trên Colab hoặc Kaggle, hãy cài đặt thư viện statsmodels:

```
if "google.colab" in sys.modules:
    %pip install -q -U statsmodels
```

```
from statsmodels.tsa.arima.model import ARIMA

origin, today = "2019-01-01", "2019-05-31"
rail_series = df.loc[origin:today]["rail"].asfreq("D")
model = ARIMA(rail_series,
               order=(1, 0, 0),
               seasonal_order=(0, 1, 1, 7))
model = model.fit()
y_pred = model.forecast() # returns 427,758.6
```

```
y_pred[0] # ARIMA forecast
```

```
427758.62631318445
```

```
df["rail"].loc["2019-06-01"] # target value
```

```
379044
```

```
df["rail"].loc["2019-05-25"] # naive forecast (value from one week earlier)
```

```
426932
```

**Cảnh báo:** Việc truy cập Pandas `Series` trực tiếp bằng chỉ mục (index) đã bị loại bỏ, vì vậy tôi đã thay thế `model.forecast()[0]` bằng `model.forecast().iloc[0]`:

```
origin, start_date, end_date = "2019-01-01", "2019-03-01", "2019-05-31"
time_period = pd.date_range(start_date, end_date)
rail_series = df.loc[origin:end_date]["rail"].asfreq("D")
y_preds = []
for today in time_period.shift(-1):
    model = ARIMA(rail_series[origin:today], # train on data up to "today"
                  order=(1, 0, 0),
                  seasonal_order=(0, 1, 1, 7))
    model = model.fit() # note that we retrain the model every day!
    y_pred = model.forecast().iloc[0]
```

```

y_preds.append(y_pred)

y_preds = pd.Series(y_preds, index=time_period)
mae = (y_preds - rail_series[time_period]).abs().mean() # returns 32,040.7

```

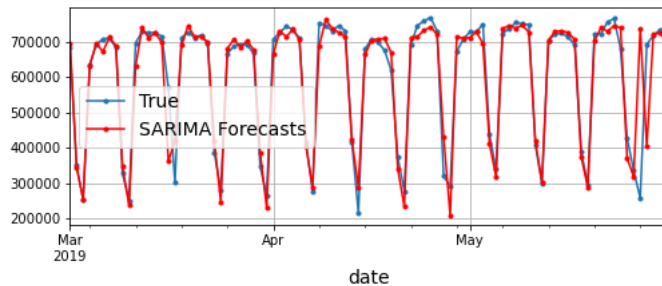
mae

32040.72008847262

```

# extra code - displays the SARIMA forecasts
fig, ax = plt.subplots(figsize=(8, 3))
rail_series.loc[time_period].plot(label="True", ax=ax, marker=".", grid=True)
ax.plot(y_preds, color="r", marker=".", label="SARIMA Forecasts")
plt.legend()
plt.show()

```



```

# extra code - shows how to plot the Autocorrelation Function (ACF) and the
# Partial Autocorrelation Function (PACF)

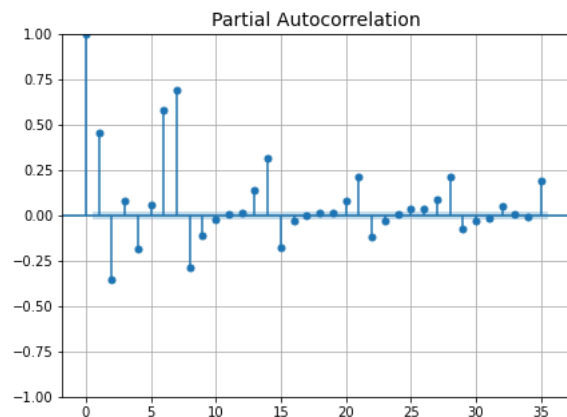
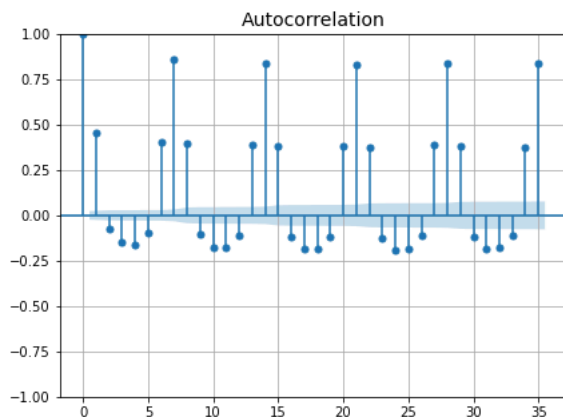
```

```

from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

fig, axs = plt.subplots(nrows=1, ncols=2, figsize=(15, 5))
plot_acf(df[period]["rail"], ax=axs[0], lags=35)
axs[0].grid()
plot_pacf(df[period]["rail"], ax=axs[1], lags=35, method="yw")
axs[1].grid()
plt.show()

```



```

import tensorflow as tf

my_series = [0, 1, 2, 3, 4, 5]
my_dataset = tf.keras.utils.timeseries_dataset_from_array(
    my_series,
    targets=my_series[3:], # the targets are 3 steps into the future
    sequence_length=3,
    batch_size=2
)
list(my_dataset)

```

2022-02-17 19:19:46.679147: I tensorflow/core/platform/cpu\_feature\_guard.cc:151] This TensorFlow binary is optimized with oneAPI To enable them in other operations, rebuild TensorFlow with the appropriate compiler flags.

```

[(<tf.Tensor: shape=(2, 3), dtype=int32, numpy=
  array([[0, 1, 2],
        [1, 2, 3]], dtype=int32)>,

```

```
<tf.Tensor: shape=(2,), dtype=int32, numpy=array([3, 4], dtype=int32)>),
(<tf.Tensor: shape=(1, 3), dtype=int32, numpy=array([[2, 3, 4]], dtype=int32)>,
<tf.Tensor: shape=(1,), dtype=int32, numpy=array([5], dtype=int32)>)]
```

```
for window_dataset in tf.data.Dataset.range(6).window(4, shift=1):
    for element in window_dataset:
        print(f"{element}", end=" ")
    print()
```

```
0 1 2 3
1 2 3 4
2 3 4 5
3 4 5
4 5
5
```

2022-02-17 19:19:46.784180: W tensorflow/core/framework/dataset.cc:744] Input of Window will not be optimized because the dataset is not a tf.data.Dataset object.

```
dataset = tf.data.Dataset.range(6).window(4, shift=1, drop_remainder=True)
dataset = dataset.flat_map(lambda window_dataset: window_dataset.batch(4))
for window_tensor in dataset:
    print(f"{window_tensor}")
```

```
[0 1 2 3]
[1 2 3 4]
[2 3 4 5]
```

```
def to_windows(dataset, length):
    dataset = dataset.window(length, shift=1, drop_remainder=True)
    return dataset.flat_map(lambda window_ds: window_ds.batch(length))
```

```
dataset = to_windows(tf.data.Dataset.range(6), 4)
dataset = dataset.map(lambda window: (window[:-1], window[-1]))
list(dataset.batch(2))
```

```
[(<tf.Tensor: shape=(2, 3), dtype=int64, numpy=
array([[0, 1, 2],
       [1, 2, 3]]>),
<tf.Tensor: shape=(2,), dtype=int64, numpy=array([3, 4])>),
(<tf.Tensor: shape=(1, 3), dtype=int64, numpy=array([[2, 3, 4]])>,
<tf.Tensor: shape=(1,), dtype=int64, numpy=array([5])>)]
```

Trước khi tiếp tục xem xét dữ liệu, hãy chia chuỗi thời gian thành ba giai đoạn: huấn luyện (training), kiểm định (validation) và kiểm tra (testing). Chúng ta sẽ chưa xem xét dữ liệu kiểm tra lúc này:

```
rail_train = df["rail"]["2016-01":"2018-12"] / 1e6
rail_valid = df["rail"]["2019-01":"2019-05"] / 1e6
rail_test = df["rail"]["2019-06:"] / 1e6
```

```
seq_length = 56
tf.random.set_seed(42) # extra code - ensures reproducibility
train_ds = tf.keras.utils.timeseries_dataset_from_array(
    rail_train.to_numpy(),
    targets=rail_train[seq_length:],
    sequence_length=seq_length,
    batch_size=32,
    shuffle=True,
    seed=42
)
valid_ds = tf.keras.utils.timeseries_dataset_from_array(
    rail_valid.to_numpy(),
    targets=rail_valid[seq_length:],
    sequence_length=seq_length,
    batch_size=32
)
```

```
tf.random.set_seed(42)
model = tf.keras.Sequential([
    tf.keras.layers.Dense(1, input_shape=[seq_length])
])
early_stopping_cb = tf.keras.callbacks.EarlyStopping(
    monitor="val_mae", patience=50, restore_best_weights=True)
opt = tf.keras.optimizers.SGD(learning_rate=0.02, momentum=0.9)
```

```
model.compile(loss=tf.keras.losses.Huber(), optimizer=opt, metrics=["mae"])
history = model.fit(train_ds, validation_data=valid_ds, epochs=500,
                    callbacks=[early_stopping_cb])
```

```
Epoch 1/500
33/33 [=====] - 0s 5ms/step - loss: 0.0098 - mae: 0.1118 - val_loss: 0.0071 - val_mae: 0.0966
Epoch 2/500
33/33 [=====] - 0s 3ms/step - loss: 0.0070 - mae: 0.0883 - val_loss: 0.0052 - val_mae: 0.0768
Epoch 3/500
33/33 [=====] - 0s 3ms/step - loss: 0.0059 - mae: 0.0796 - val_loss: 0.0050 - val_mae: 0.0741
Epoch 4/500
33/33 [=====] - 0s 3ms/step - loss: 0.0055 - mae: 0.0761 - val_loss: 0.0049 - val_mae: 0.0732
Epoch 5/500
33/33 [=====] - 0s 3ms/step - loss: 0.0054 - mae: 0.0749 - val_loss: 0.0043 - val_mae: 0.0666
Epoch 6/500
33/33 [=====] - 0s 3ms/step - loss: 0.0051 - mae: 0.0724 - val_loss: 0.0041 - val_mae: 0.0638
Epoch 7/500
33/33 [=====] - 0s 3ms/step - loss: 0.0047 - mae: 0.0696 - val_loss: 0.0040 - val_mae: 0.0615
Epoch 8/500
33/33 [=====] - 0s 3ms/step - loss: 0.0051 - mae: 0.0735 - val_loss: 0.0038 - val_mae: 0.0599
Epoch 9/500
33/33 [=====] - 0s 3ms/step - loss: 0.0045 - mae: 0.0670 - val_loss: 0.0037 - val_mae: 0.0599
Epoch 10/500
33/33 [=====] - 0s 3ms/step - loss: 0.0046 - mae: 0.0677 - val_loss: 0.0041 - val_mae: 0.0658
Epoch 11/500
33/33 [=====] - 0s 3ms/step - loss: 0.0044 - mae: 0.0664 - val_loss: 0.0038 - val_mae: 0.0611
Epoch 12/500
33/33 [=====] - 0s 3ms/step - loss: 0.0042 - mae: 0.0634 - val_loss: 0.0034 - val_mae: 0.0551
Epoch 13/500
33/33 [=====] - 0s 3ms/step - loss: 0.0046 - mae: 0.0680 - val_loss: 0.0056 - val_mae: 0.0829
Epoch 14/500
33/33 [=====] - 0s 3ms/step - loss: 0.0044 - mae: 0.0671 - val_loss: 0.0039 - val_mae: 0.0637
Epoch 15/500
33/33 [=====] - 0s 3ms/step - loss: 0.0044 - mae: 0.0673 - val_loss: 0.0037 - val_mae: 0.0610
Epoch 16/500
33/33 [=====] - 0s 4ms/step - loss: 0.0045 - mae: 0.0676 - val_loss: 0.0035 - val_mae: 0.0584
Epoch 17/500
33/33 [=====] - 0s 3ms/step - loss: 0.0044 - mae: 0.0662 - val_loss: 0.0033 - val_mae: 0.0544
Epoch 18/500
<<396 more lines>>
33/33 [=====] - 0s 3ms/step - loss: 0.0026 - mae: 0.0440 - val_loss: 0.0023 - val_mae: 0.0404
Epoch 217/500
33/33 [=====] - 0s 3ms/step - loss: 0.0029 - mae: 0.0500 - val_loss: 0.0028 - val_mae: 0.0526
Epoch 218/500
33/33 [=====] - 0s 3ms/step - loss: 0.0026 - mae: 0.0458 - val_loss: 0.0023 - val_mae: 0.0387
Epoch 219/500
33/33 [=====] - 0s 3ms/step - loss: 0.0027 - mae: 0.0454 - val_loss: 0.0023 - val_mae: 0.0396
Epoch 220/500
33/33 [=====] - 0s 3ms/step - loss: 0.0026 - mae: 0.0444 - val_loss: 0.0026 - val_mae: 0.0425
Epoch 221/500
33/33 [=====] - 0s 3ms/step - loss: 0.0026 - mae: 0.0452 - val_loss: 0.0023 - val_mae: 0.0387
Epoch 222/500
33/33 [=====] - 0s 3ms/step - loss: 0.0025 - mae: 0.0433 - val_loss: 0.0024 - val_mae: 0.0432
Epoch 223/500
33/33 [=====] - 0s 3ms/step - loss: 0.0026 - mae: 0.0441 - val_loss: 0.0029 - val_mae: 0.0489
Epoch 224/500
33/33 [=====] - 0s 3ms/step - loss: 0.0031 - mae: 0.0524 - val_loss: 0.0023 - val_mae: 0.0394
Epoch 225/500
33/33 [=====] - 0s 3ms/step - loss: 0.0025 - mae: 0.0424 - val_loss: 0.0023 - val_mae: 0.0386
Epoch 226/500
33/33 [=====] - 0s 3ms/step - loss: 0.0026 - mae: 0.0438 - val_loss: 0.0023 - val_mae: 0.0383
Epoch 227/500
```

```
# extra code - evaluates the model
valid_loss, valid_mae = model.evaluate(valid_ds)
valid_mae * 1e6
```

```
3/3 [=====] - 0s 2ms/step - loss: 0.0022 - mae: 0.0379
37866.38006567955
```

## ✧ Sử dụng một Simple RNN

```
tf.random.set_seed(42) # extra code - ensures reproducibility
model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(1, input_shape=[None, 1])
])
```

```
# extra code - defines a utility function we'll reuse several time
```

```
def fit_and_evaluate(model, train_set, valid_set, learning_rate, epochs=500):
    early_stopping_cb = tf.keras.callbacks.EarlyStopping(
        monitor="val_mae", patience=50, restore_best_weights=True)
    opt = tf.keras.optimizers.SGD(learning_rate=learning_rate, momentum=0.9)
    model.compile(loss=tf.keras.losses.Huber(), optimizer=opt, metrics=["mae"])
    history = model.fit(train_set, validation_data=valid_set, epochs=epochs,
        callbacks=[early_stopping_cb])
    valid_loss, valid_mae = model.evaluate(valid_set)
    return valid_mae * 1e6
```

```
fit_and_evaluate(model, train_ds, valid_ds, learning_rate=0.02)
```

```
Epoch 1/500
33/33 [=====] - 1s 11ms/step - loss: 0.0219 - mae: 0.1637 - val_loss: 0.0195 - val_mae: 0.1394
Epoch 2/500
33/33 [=====] - 0s 7ms/step - loss: 0.0170 - mae: 0.1553 - val_loss: 0.0179 - val_mae: 0.1482
Epoch 3/500
33/33 [=====] - 0s 7ms/step - loss: 0.0166 - mae: 0.1555 - val_loss: 0.0176 - val_mae: 0.1501
Epoch 4/500
33/33 [=====] - 0s 7ms/step - loss: 0.0164 - mae: 0.1558 - val_loss: 0.0173 - val_mae: 0.1534
Epoch 5/500
33/33 [=====] - 0s 7ms/step - loss: 0.0163 - mae: 0.1572 - val_loss: 0.0172 - val_mae: 0.1479
Epoch 6/500
33/33 [=====] - 0s 7ms/step - loss: 0.0162 - mae: 0.1555 - val_loss: 0.0170 - val_mae: 0.1496
Epoch 7/500
33/33 [=====] - 0s 7ms/step - loss: 0.0162 - mae: 0.1556 - val_loss: 0.0168 - val_mae: 0.1552
Epoch 8/500
33/33 [=====] - 0s 7ms/step - loss: 0.0161 - mae: 0.1580 - val_loss: 0.0169 - val_mae: 0.1448
Epoch 9/500
33/33 [=====] - 0s 7ms/step - loss: 0.0160 - mae: 0.1563 - val_loss: 0.0168 - val_mae: 0.1451
Epoch 10/500
33/33 [=====] - 0s 7ms/step - loss: 0.0159 - mae: 0.1562 - val_loss: 0.0167 - val_mae: 0.1454
Epoch 11/500
33/33 [=====] - 0s 7ms/step - loss: 0.0159 - mae: 0.1564 - val_loss: 0.0164 - val_mae: 0.1491
Epoch 12/500
33/33 [=====] - 0s 7ms/step - loss: 0.0158 - mae: 0.1559 - val_loss: 0.0165 - val_mae: 0.1445
Epoch 13/500
33/33 [=====] - 0s 8ms/step - loss: 0.0158 - mae: 0.1556 - val_loss: 0.0162 - val_mae: 0.1514
Epoch 14/500
33/33 [=====] - 0s 7ms/step - loss: 0.0157 - mae: 0.1564 - val_loss: 0.0162 - val_mae: 0.1533
Epoch 15/500
33/33 [=====] - 0s 8ms/step - loss: 0.0157 - mae: 0.1553 - val_loss: 0.0165 - val_mae: 0.1420
Epoch 16/500
33/33 [=====] - 0s 8ms/step - loss: 0.0158 - mae: 0.1562 - val_loss: 0.0164 - val_mae: 0.1425
Epoch 17/500
33/33 [=====] - 0s 8ms/step - loss: 0.0156 - mae: 0.1570 - val_loss: 0.0164 - val_mae: 0.1407
Epoch 18/500
<<687 more lines>>
Epoch 362/500
33/33 [=====] - 0s 8ms/step - loss: 0.0103 - mae: 0.1130 - val_loss: 0.0103 - val_mae: 0.1029
Epoch 363/500
33/33 [=====] - 0s 7ms/step - loss: 0.0103 - mae: 0.1128 - val_loss: 0.0103 - val_mae: 0.1029
Epoch 364/500
33/33 [=====] - 0s 8ms/step - loss: 0.0104 - mae: 0.1131 - val_loss: 0.0102 - val_mae: 0.1029
Epoch 365/500
33/33 [=====] - 0s 7ms/step - loss: 0.0104 - mae: 0.1133 - val_loss: 0.0103 - val_mae: 0.1029
Epoch 366/500
33/33 [=====] - 0s 7ms/step - loss: 0.0104 - mae: 0.1128 - val_loss: 0.0103 - val_mae: 0.1028
Epoch 367/500
33/33 [=====] - 0s 8ms/step - loss: 0.0103 - mae: 0.1129 - val_loss: 0.0103 - val_mae: 0.1029
Epoch 368/500
33/33 [=====] - 0s 7ms/step - loss: 0.0104 - mae: 0.1135 - val_loss: 0.0102 - val_mae: 0.1030
Epoch 369/500
33/33 [=====] - 0s 7ms/step - loss: 0.0103 - mae: 0.1129 - val_loss: 0.0103 - val_mae: 0.1028
Epoch 370/500
33/33 [=====] - 0s 7ms/step - loss: 0.0104 - mae: 0.1129 - val_loss: 0.0103 - val_mae: 0.1029
Epoch 371/500
33/33 [=====] - 0s 7ms/step - loss: 0.0103 - mae: 0.1130 - val_loss: 0.0103 - val_mae: 0.1029
Epoch 372/500
33/33 [=====] - 0s 7ms/step - loss: 0.0103 - mae: 0.1131 - val_loss: 0.0103 - val_mae: 0.1029
```

```
tf.random.set_seed(42) # extra code - ensures reproducibility
univar_model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(32, input_shape=[None, 1]),
    tf.keras.layers.Dense(1) # no activation function by default
])
```

```
# extra code - compiles, fits, and evaluates the model, like earlier
fit_and_evaluate(univar_model, train_ds, valid_ds, learning_rate=0.05)
```



```

Epoch 1/500
33/33 [=====] - 1s 13ms/step - loss: 0.0489 - mae: 0.2061 - val_loss: 0.0060 - val_mae: 0.0854
Epoch 2/500
33/33 [=====] - 0s 9ms/step - loss: 0.0060 - mae: 0.0813 - val_loss: 0.0052 - val_mae: 0.0825
Epoch 3/500
33/33 [=====] - 0s 9ms/step - loss: 0.0042 - mae: 0.0647 - val_loss: 0.0041 - val_mae: 0.0656
Epoch 4/500
33/33 [=====] - 0s 9ms/step - loss: 0.0041 - mae: 0.0636 - val_loss: 0.0042 - val_mae: 0.0714
Epoch 5/500
33/33 [=====] - 0s 10ms/step - loss: 0.0039 - mae: 0.0595 - val_loss: 0.0023 - val_mae: 0.0387
Epoch 6/500
33/33 [=====] - 0s 9ms/step - loss: 0.0033 - mae: 0.0542 - val_loss: 0.0026 - val_mae: 0.0423
Epoch 7/500
33/33 [=====] - 0s 9ms/step - loss: 0.0032 - mae: 0.0502 - val_loss: 0.0021 - val_mae: 0.0354
Epoch 8/500
33/33 [=====] - 0s 9ms/step - loss: 0.0030 - mae: 0.0500 - val_loss: 0.0020 - val_mae: 0.0345
Epoch 9/500
33/33 [=====] - 0s 9ms/step - loss: 0.0033 - mae: 0.0539 - val_loss: 0.0050 - val_mae: 0.0825
Epoch 10/500
33/33 [=====] - 0s 9ms/step - loss: 0.0034 - mae: 0.0573 - val_loss: 0.0023 - val_mae: 0.0399
Epoch 11/500
33/33 [=====] - 0s 9ms/step - loss: 0.0030 - mae: 0.0493 - val_loss: 0.0022 - val_mae: 0.0377
Epoch 12/500
33/33 [=====] - 0s 9ms/step - loss: 0.0029 - mae: 0.0478 - val_loss: 0.0019 - val_mae: 0.0328
Epoch 13/500
33/33 [=====] - 0s 9ms/step - loss: 0.0028 - mae: 0.0460 - val_loss: 0.0024 - val_mae: 0.0404
Epoch 14/500
33/33 [=====] - 0s 9ms/step - loss: 0.0029 - mae: 0.0487 - val_loss: 0.0022 - val_mae: 0.0371
Epoch 15/500
33/33 [=====] - 0s 9ms/step - loss: 0.0029 - mae: 0.0469 - val_loss: 0.0019 - val_mae: 0.0306
Epoch 16/500
33/33 [=====] - 0s 9ms/step - loss: 0.0027 - mae: 0.0465 - val_loss: 0.0019 - val_mae: 0.0348
Epoch 17/500
33/33 [=====] - 0s 10ms/step - loss: 0.0029 - mae: 0.0485 - val_loss: 0.0024 - val_mae: 0.0426
Epoch 18/500
<<201 more lines>>
Epoch 119/500
33/33 [=====] - 0s 10ms/step - loss: 0.0024 - mae: 0.0428 - val_loss: 0.0020 - val_mae: 0.0334
Epoch 120/500
33/33 [=====] - 0s 9ms/step - loss: 0.0024 - mae: 0.0423 - val_loss: 0.0019 - val_mae: 0.0362
Epoch 121/500
33/33 [=====] - 0s 11ms/step - loss: 0.0023 - mae: 0.0408 - val_loss: 0.0019 - val_mae: 0.0356
Epoch 122/500
33/33 [=====] - 0s 10ms/step - loss: 0.0023 - mae: 0.0397 - val_loss: 0.0020 - val_mae: 0.0395
Epoch 123/500
33/33 [=====] - 0s 10ms/step - loss: 0.0024 - mae: 0.0429 - val_loss: 0.0017 - val_mae: 0.0297
Epoch 124/500
33/33 [=====] - 0s 9ms/step - loss: 0.0025 - mae: 0.0437 - val_loss: 0.0019 - val_mae: 0.0359
Epoch 125/500
33/33 [=====] - 0s 9ms/step - loss: 0.0024 - mae: 0.0430 - val_loss: 0.0017 - val_mae: 0.0305
Epoch 126/500
33/33 [=====] - 0s 8ms/step - loss: 0.0023 - mae: 0.0399 - val_loss: 0.0021 - val_mae: 0.0409
Epoch 127/500
33/33 [=====] - 0s 9ms/step - loss: 0.0023 - mae: 0.0411 - val_loss: 0.0018 - val_mae: 0.0314
Epoch 128/500
33/33 [=====] - 0s 9ms/step - loss: 0.0023 - mae: 0.0394 - val_loss: 0.0021 - val_mae: 0.0392
Epoch 129/500
33/33 [=====] - 0s 10ms/step - loss: 0.0023 - mae: 0.0416 - val_loss: 0.0017 - val_mae: 0.0329

```

## Deep RNNs (RNN sâu)

```

tf.random.set_seed(42) # extra code - ensures reproducibility
deep_model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(32, return_sequences=True, input_shape=[None, 1]),
    tf.keras.layers.SimpleRNN(32, return_sequences=True),
    tf.keras.layers.SimpleRNN(32),
    tf.keras.layers.Dense(1)
])

```

```

# extra code - compiles, fits, and evaluates the model, like earlier
fit_and_evaluate(deep_model, train_ds, valid_ds, learning_rate=0.01)

```

```

Epoch 1/500
33/33 [=====] - 2s 32ms/step - loss: 0.0393 - mae: 0.2109 - val_loss: 0.0085 - val_mae: 0.1110
Epoch 2/500
33/33 [=====] - 1s 25ms/step - loss: 0.0068 - mae: 0.0858 - val_loss: 0.0032 - val_mae: 0.0629
Epoch 3/500
33/33 [=====] - 1s 24ms/step - loss: 0.0055 - mae: 0.0750 - val_loss: 0.0035 - val_mae: 0.0638
Epoch 4/500
33/33 [=====] - 1s 27ms/step - loss: 0.0048 - mae: 0.0678 - val_loss: 0.0021 - val_mae: 0.0429

```

```

Epoch 5/500
33/33 [=====] - 1s 27ms/step - loss: 0.0043 - mae: 0.0606 - val_loss: 0.0020 - val_mae: 0.0408
Epoch 6/500
33/33 [=====] - 1s 27ms/step - loss: 0.0042 - mae: 0.0591 - val_loss: 0.0027 - val_mae: 0.0502
Epoch 7/500
33/33 [=====] - 1s 25ms/step - loss: 0.0045 - mae: 0.0635 - val_loss: 0.0025 - val_mae: 0.0469
Epoch 8/500
33/33 [=====] - 1s 24ms/step - loss: 0.0042 - mae: 0.0592 - val_loss: 0.0027 - val_mae: 0.0498
Epoch 9/500
33/33 [=====] - 1s 26ms/step - loss: 0.0039 - mae: 0.0555 - val_loss: 0.0034 - val_mae: 0.0619
Epoch 10/500
33/33 [=====] - 1s 25ms/step - loss: 0.0041 - mae: 0.0590 - val_loss: 0.0022 - val_mae: 0.0400
Epoch 11/500
33/33 [=====] - 1s 25ms/step - loss: 0.0037 - mae: 0.0526 - val_loss: 0.0022 - val_mae: 0.0408
Epoch 12/500
33/33 [=====] - 1s 26ms/step - loss: 0.0037 - mae: 0.0543 - val_loss: 0.0019 - val_mae: 0.0349
Epoch 13/500
33/33 [=====] - 1s 23ms/step - loss: 0.0034 - mae: 0.0493 - val_loss: 0.0019 - val_mae: 0.0334
Epoch 14/500
33/33 [=====] - 1s 23ms/step - loss: 0.0035 - mae: 0.0505 - val_loss: 0.0020 - val_mae: 0.0341
Epoch 15/500
33/33 [=====] - 1s 23ms/step - loss: 0.0034 - mae: 0.0494 - val_loss: 0.0020 - val_mae: 0.0360
Epoch 16/500
33/33 [=====] - 1s 23ms/step - loss: 0.0033 - mae: 0.0496 - val_loss: 0.0027 - val_mae: 0.0474
Epoch 17/500
33/33 [=====] - 1s 23ms/step - loss: 0.0037 - mae: 0.0559 - val_loss: 0.0020 - val_mae: 0.0332
Epoch 18/500
<<103 more lines>>
Epoch 70/500
33/33 [=====] - 1s 24ms/step - loss: 0.0026 - mae: 0.0422 - val_loss: 0.0022 - val_mae: 0.0363
Epoch 71/500
33/33 [=====] - 1s 24ms/step - loss: 0.0027 - mae: 0.0458 - val_loss: 0.0019 - val_mae: 0.0321
Epoch 72/500
33/33 [=====] - 1s 24ms/step - loss: 0.0025 - mae: 0.0413 - val_loss: 0.0020 - val_mae: 0.0335
Epoch 73/500
33/33 [=====] - 1s 24ms/step - loss: 0.0026 - mae: 0.0435 - val_loss: 0.0021 - val_mae: 0.0354
Epoch 74/500
33/33 [=====] - 1s 25ms/step - loss: 0.0026 - mae: 0.0436 - val_loss: 0.0021 - val_mae: 0.0357
Epoch 75/500
33/33 [=====] - 1s 24ms/step - loss: 0.0026 - mae: 0.0432 - val_loss: 0.0021 - val_mae: 0.0347
Epoch 76/500
33/33 [=====] - 1s 24ms/step - loss: 0.0025 - mae: 0.0421 - val_loss: 0.0027 - val_mae: 0.0477
Epoch 77/500
33/33 [=====] - 1s 24ms/step - loss: 0.0027 - mae: 0.0444 - val_loss: 0.0019 - val_mae: 0.0320
Epoch 78/500
33/33 [=====] - 1s 24ms/step - loss: 0.0028 - mae: 0.0468 - val_loss: 0.0019 - val_mae: 0.0318
Epoch 79/500
33/33 [=====] - 1s 24ms/step - loss: 0.0027 - mae: 0.0466 - val_loss: 0.0021 - val_mae: 0.0366
Epoch 80/500
33/33 [=====] - 1s 24ms/step - loss: 0.0026 - mae: 0.0442 - val_loss: 0.0025 - val_mae: 0.0454

```

## ✓ Chuỗi thời gian đa biến (Multivariate time series)

**Cảnh báo:** trong các phiên bản Pandas gần đây, `get_dummies()` trả về kiểu boolean thay vì float, vì vậy tôi đã thêm `dtype=float` để đảm bảo chúng ta nhận được kiểu số thực.

```

df_mulvar = df[["bus", "rail"]] / 1e6 # use both bus & rail series as input
df_mulvar["next_day_type"] = df["day_type"].shift(-1) # we know tomorrow's type
df_mulvar = pd.get_dummies(df_mulvar, dtype=float) # one-hot encode the day type

```

```

mulvar_train = df_mulvar["2016-01":"2018-12"]
mulvar_valid = df_mulvar["2019-01":"2019-05"]
mulvar_test = df_mulvar["2019-06":]

```

```

tf.random.set_seed(42) # extra code - ensures reproducibility

train_mulvar_ds = tf.keras.utils.timeseries_dataset_from_array(
    mulvar_train.to_numpy(), # use all 5 columns as input
    targets=mulvar_train["rail"][seq_length:], # forecast only the rail series
    sequence_length=seq_length,
    batch_size=32,
    shuffle=True,
    seed=42
)

valid_mulvar_ds = tf.keras.utils.timeseries_dataset_from_array(
    mulvar_valid.to_numpy(),
    targets=mulvar_valid["rail"][seq_length:],

```

```
sequence_length=seq_length,
batch_size=32
)
```

```
tf.random.set_seed(42) # extra code - ensures reproducibility
mulvar_model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(32, input_shape=[None, 5]),
    tf.keras.layers.Dense(1)
])
```

```
# extra code - compiles, fits, and evaluates the model, like earlier
fit_and_evaluate(mulvar_model, train_mulvar_ds, valid_mulvar_ds,
                 learning_rate=0.05)
```

```
Epoch 1/500
33/33 [=====] - 1s 17ms/step - loss: 0.0386 - mae: 0.1872 - val_loss: 0.0011 - val_mae: 0.0346
Epoch 2/500
33/33 [=====] - 0s 11ms/step - loss: 0.0029 - mae: 0.0585 - val_loss: 0.0040 - val_mae: 0.0790
Epoch 3/500
33/33 [=====] - 0s 10ms/step - loss: 0.0018 - mae: 0.0435 - val_loss: 7.7056e-04 - val_mae: 0.0273
Epoch 4/500
33/33 [=====] - 0s 10ms/step - loss: 0.0017 - mae: 0.0407 - val_loss: 0.0010 - val_mae: 0.0362
Epoch 5/500
33/33 [=====] - 0s 10ms/step - loss: 0.0015 - mae: 0.0386 - val_loss: 8.1681e-04 - val_mae: 0.0306
Epoch 6/500
33/33 [=====] - 0s 9ms/step - loss: 0.0014 - mae: 0.0372 - val_loss: 0.0011 - val_mae: 0.0380
Epoch 7/500
33/33 [=====] - 0s 10ms/step - loss: 0.0014 - mae: 0.0366 - val_loss: 7.9942e-04 - val_mae: 0.0289
Epoch 8/500
33/33 [=====] - 0s 10ms/step - loss: 0.0013 - mae: 0.0344 - val_loss: 6.9211e-04 - val_mae: 0.0271
Epoch 9/500
33/33 [=====] - 0s 9ms/step - loss: 0.0015 - mae: 0.0374 - val_loss: 8.2185e-04 - val_mae: 0.0299
Epoch 10/500
33/33 [=====] - 0s 10ms/step - loss: 0.0014 - mae: 0.0363 - val_loss: 0.0017 - val_mae: 0.0494
Epoch 11/500
33/33 [=====] - 0s 10ms/step - loss: 0.0013 - mae: 0.0357 - val_loss: 0.0016 - val_mae: 0.0473
Epoch 12/500
33/33 [=====] - 0s 10ms/step - loss: 0.0013 - mae: 0.0337 - val_loss: 8.0260e-04 - val_mae: 0.0287
Epoch 13/500
33/33 [=====] - 0s 10ms/step - loss: 0.0013 - mae: 0.0349 - val_loss: 0.0011 - val_mae: 0.0389
Epoch 14/500
33/33 [=====] - 0s 9ms/step - loss: 0.0014 - mae: 0.0363 - val_loss: 6.3723e-04 - val_mae: 0.0245
Epoch 15/500
33/33 [=====] - 0s 10ms/step - loss: 0.0012 - mae: 0.0340 - val_loss: 6.2749e-04 - val_mae: 0.0255
Epoch 16/500
33/33 [=====] - 0s 9ms/step - loss: 0.0013 - mae: 0.0342 - val_loss: 0.0020 - val_mae: 0.0549
Epoch 17/500
33/33 [=====] - 0s 9ms/step - loss: 0.0012 - mae: 0.0332 - val_loss: 7.3463e-04 - val_mae: 0.0275
Epoch 18/500
<<181 more lines>>
Epoch 109/500
33/33 [=====] - 0s 9ms/step - loss: 0.0011 - mae: 0.0319 - val_loss: 6.3961e-04 - val_mae: 0.0244
Epoch 110/500
33/33 [=====] - 0s 9ms/step - loss: 0.0012 - mae: 0.0354 - val_loss: 0.0013 - val_mae: 0.0433
Epoch 111/500
33/33 [=====] - 0s 9ms/step - loss: 0.0010 - mae: 0.0307 - val_loss: 7.3263e-04 - val_mae: 0.0281
Epoch 112/500
33/33 [=====] - 0s 10ms/step - loss: 0.0014 - mae: 0.0377 - val_loss: 7.8642e-04 - val_mae: 0.0293
Epoch 113/500
33/33 [=====] - 0s 9ms/step - loss: 0.0012 - mae: 0.0340 - val_loss: 0.0013 - val_mae: 0.0415
Epoch 114/500
33/33 [=====] - 0s 9ms/step - loss: 0.0012 - mae: 0.0344 - val_loss: 0.0011 - val_mae: 0.0376
Epoch 115/500
33/33 [=====] - 0s 9ms/step - loss: 0.0011 - mae: 0.0314 - val_loss: 0.0010 - val_mae: 0.0344
Epoch 116/500
33/33 [=====] - 0s 9ms/step - loss: 0.0013 - mae: 0.0374 - val_loss: 7.2942e-04 - val_mae: 0.0264
Epoch 117/500
33/33 [=====] - 0s 9ms/step - loss: 0.0011 - mae: 0.0336 - val_loss: 0.0011 - val_mae: 0.0393
Epoch 118/500
33/33 [=====] - 0s 9ms/step - loss: 0.0014 - mae: 0.0392 - val_loss: 0.0015 - val_mae: 0.0455
Epoch 119/500
33/33 [=====] - 0s 9ms/step - loss: 0.0012 - mae: 0.0369 - val_loss: 0.0011 - val_mae: 0.0363
```

```
# extra code - build and train a multitask RNN that forecasts both bus and rail
```

```
tf.random.set_seed(42)

seq_length = 56
train_multitask_ds = tf.keras.utils.timeseries_dataset_from_array(
    mulvar_train.to_numpy(),
```

```

        targets=mulvar_train[["bus", "rail"]][seq_length:], # 2 targets per day
        sequence_length=seq_length,
        batch_size=32,
        shuffle=True,
        seed=42
    )
    valid_multitask_ds = tf.keras.utils.timeseries_dataset_from_array(
        mulvar_valid.to_numpy(),
        targets=mulvar_valid[["bus", "rail"]][seq_length:],
        sequence_length=seq_length,
        batch_size=32
    )

    tf.random.set_seed(42)
    multitask_model = tf.keras.Sequential([
        tf.keras.layers.SimpleRNN(32, input_shape=[None, 5]),
        tf.keras.layers.Dense(2)
    ])

    fit_and_evaluate(multitask_model, train_multitask_ds, valid_multitask_ds,
                     learning_rate=0.02)

```

```

Epoch 1/500
33/33 [=====] - 1s 13ms/step - loss: 0.0398 - mae: 0.1953 - val_loss: 0.0073 - val_mae: 0.0998
Epoch 2/500
33/33 [=====] - 0s 9ms/step - loss: 0.0039 - mae: 0.0632 - val_loss: 0.0012 - val_mae: 0.0384
Epoch 3/500
33/33 [=====] - 0s 9ms/step - loss: 0.0027 - mae: 0.0509 - val_loss: 0.0010 - val_mae: 0.0362
Epoch 4/500
33/33 [=====] - 0s 9ms/step - loss: 0.0024 - mae: 0.0488 - val_loss: 0.0018 - val_mae: 0.0491
Epoch 5/500
33/33 [=====] - 0s 10ms/step - loss: 0.0023 - mae: 0.0473 - val_loss: 0.0012 - val_mae: 0.0372
Epoch 6/500
33/33 [=====] - 0s 9ms/step - loss: 0.0022 - mae: 0.0463 - val_loss: 0.0011 - val_mae: 0.0361
Epoch 7/500
33/33 [=====] - 0s 9ms/step - loss: 0.0019 - mae: 0.0442 - val_loss: 8.8553e-04 - val_mae: 0.0322
Epoch 8/500
33/33 [=====] - 0s 9ms/step - loss: 0.0018 - mae: 0.0427 - val_loss: 9.3772e-04 - val_mae: 0.0339
Epoch 9/500
33/33 [=====] - 0s 9ms/step - loss: 0.0017 - mae: 0.0411 - val_loss: 9.0027e-04 - val_mae: 0.0324
Epoch 10/500
33/33 [=====] - 0s 10ms/step - loss: 0.0019 - mae: 0.0440 - val_loss: 0.0014 - val_mae: 0.0427
Epoch 11/500
33/33 [=====] - 0s 10ms/step - loss: 0.0017 - mae: 0.0415 - val_loss: 0.0021 - val_mae: 0.0546
Epoch 12/500
33/33 [=====] - 0s 9ms/step - loss: 0.0017 - mae: 0.0412 - val_loss: 8.3458e-04 - val_mae: 0.0311
Epoch 13/500
33/33 [=====] - 0s 9ms/step - loss: 0.0016 - mae: 0.0399 - val_loss: 8.2083e-04 - val_mae: 0.0311
Epoch 14/500
33/33 [=====] - 0s 10ms/step - loss: 0.0015 - mae: 0.0391 - val_loss: 0.0010 - val_mae: 0.0358
Epoch 15/500
33/33 [=====] - 0s 10ms/step - loss: 0.0016 - mae: 0.0407 - val_loss: 0.0011 - val_mae: 0.0361
Epoch 16/500
33/33 [=====] - 0s 9ms/step - loss: 0.0014 - mae: 0.0378 - val_loss: 0.0012 - val_mae: 0.0380
Epoch 17/500
33/33 [=====] - 0s 10ms/step - loss: 0.0015 - mae: 0.0394 - val_loss: 9.6802e-04 - val_mae: 0.0346
Epoch 18/500
<<215 more lines>>
Epoch 126/500
33/33 [=====] - 0s 9ms/step - loss: 0.0011 - mae: 0.0317 - val_loss: 6.8940e-04 - val_mae: 0.0271
Epoch 127/500
33/33 [=====] - 0s 9ms/step - loss: 0.0011 - mae: 0.0328 - val_loss: 0.0013 - val_mae: 0.0412
Epoch 128/500
33/33 [=====] - 0s 10ms/step - loss: 0.0012 - mae: 0.0344 - val_loss: 7.6342e-04 - val_mae: 0.0292
Epoch 129/500
33/33 [=====] - 0s 9ms/step - loss: 0.0011 - mae: 0.0328 - val_loss: 8.3261e-04 - val_mae: 0.0311
Epoch 130/500
33/33 [=====] - 0s 10ms/step - loss: 0.0011 - mae: 0.0316 - val_loss: 6.7921e-04 - val_mae: 0.0263
Epoch 131/500
33/33 [=====] - 0s 9ms/step - loss: 0.0011 - mae: 0.0320 - val_loss: 7.7970e-04 - val_mae: 0.0297
Epoch 132/500
33/33 [=====] - 0s 9ms/step - loss: 0.0011 - mae: 0.0334 - val_loss: 7.4201e-04 - val_mae: 0.0286
Epoch 133/500
33/33 [=====] - 0s 9ms/step - loss: 0.0011 - mae: 0.0330 - val_loss: 9.3328e-04 - val_mae: 0.0339
Epoch 134/500
33/33 [=====] - 0s 10ms/step - loss: 0.0011 - mae: 0.0322 - val_loss: 6.9349e-04 - val_mae: 0.0267
Epoch 135/500
33/33 [=====] - 0s 10ms/step - loss: 0.0011 - mae: 0.0317 - val_loss: 6.6078e-04 - val_mae: 0.0261
Epoch 136/500
33/33 [=====] - 0s 10ms/step - loss: 0.0011 - mae: 0.0322 - val_loss: 9.1503e-04 - val_mae: 0.0322

```

```
# extra code - evaluates the naive forecasts for bus
bus_naive = mulvar_valid["bus"].shift(7)[seq_length:]
bus_target = mulvar_valid["bus"][seq_length:]
(bus_target - bus_naive).abs().mean() * 1e6
```

43441.63157894738

```
# extra code - evaluates the multitask RNN's forecasts both bus and rail
Y_preds_valid = multitask_model.predict(valid_multitask_ds)
for idx, name in enumerate(["bus", "rail"]):
    mae = 1e6 * tf.keras.metrics.MeanAbsoluteError()(
        mulvar_valid[name][seq_length:], Y_preds_valid[:, idx])
    print(name, int(mae))
```

bus 26369  
rail 25330

## ✓ Dự báo trước nhiều bước (Forecasting Several Steps Ahead)

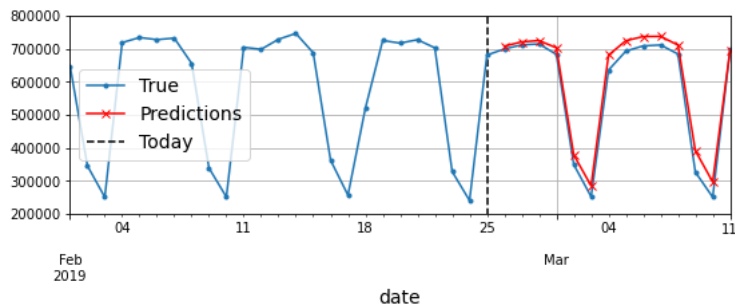
```
import numpy as np

X = rail_valid.to_numpy()[np.newaxis, :seq_length, np.newaxis]
for step_ahead in range(14):
    y_pred_one = univar_model.predict(X)
    X = np.concatenate([X, y_pred_one.reshape(1, 1, 1)], axis=1)
```

```
# extra code - generates and saves Figure 15-11

# The forecasts start on 2019-02-26, as it is the 57th day of 2019, and they end
# on 2019-03-11. That's 14 days in total.
Y_pred = pd.Series(X[0, -14:, 0],
                    index=pd.date_range("2019-02-26", "2019-03-11"))

fig, ax = plt.subplots(figsize=(8, 3.5))
(rail_valid * 1e6)["2019-02-01":"2019-03-11"].plot(
    label="True", marker=".", ax=ax)
(Y_pred * 1e6).plot(
    label="Predictions", grid=True, marker="x", color="r", ax=ax)
ax.vlines("2019-02-25", 0, 1e6, color="k", linestyle="--", label="Today")
ax.set_ylim([200_000, 800_000])
plt.legend(loc="center left")
save_fig("forecast_ahead_plot")
plt.show()
```



Bây giờ hãy tạo một RNN dự đoán tất cả 14 giá trị tiếp theo cùng một lúc:

```
tf.random.set_seed(42) # extra code - ensures reproducibility

def split_inputs_and_targets(mulvar_series, ahead=14, target_col=1):
    return mulvar_series[:, :-ahead], mulvar_series[:, -ahead:, target_col]

ahead_train_ds = tf.keras.utils.timeseries_dataset_from_array(
    mulvar_train.to_numpy(),
    targets=None,
    sequence_length=seq_length + 14,
    batch_size=32,
    shuffle=True,
    seed=42
).map(split_inputs_and_targets)
```

```

ahead_valid_ds = tf.keras.utils.timeseries_dataset_from_array(
    mulvar_valid.to_numpy(),
    targets=None,
    sequence_length=seq_length + 14,
    batch_size=32
).map(split_inputs_and_targets)

```

```
tf.random.set_seed(42)
```

```

ahead_model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(32, input_shape=[None, 5]),
    tf.keras.layers.Dense(14)
])

```

```

# extra code - compiles, fits, and evaluates the model, like earlier
fit_and_evaluate(ahead_model, ahead_train_ds, ahead_valid_ds,
    learning_rate=0.02)

```

```

Epoch 1/500
33/33 [=====] - 1s 12ms/step - loss: 0.1250 - mae: 0.3791 - val_loss: 0.0287 - val_mae: 0.1935
Epoch 2/500
33/33 [=====] - 0s 9ms/step - loss: 0.0191 - mae: 0.1613 - val_loss: 0.0136 - val_mae: 0.1289
Epoch 3/500
33/33 [=====] - 0s 8ms/step - loss: 0.0131 - mae: 0.1303 - val_loss: 0.0102 - val_mae: 0.1113
Epoch 4/500
33/33 [=====] - 0s 8ms/step - loss: 0.0108 - mae: 0.1164 - val_loss: 0.0083 - val_mae: 0.1009
Epoch 5/500
33/33 [=====] - 0s 8ms/step - loss: 0.0093 - mae: 0.1068 - val_loss: 0.0071 - val_mae: 0.0931
Epoch 6/500
33/33 [=====] - 0s 8ms/step - loss: 0.0083 - mae: 0.0996 - val_loss: 0.0061 - val_mae: 0.0862
Epoch 7/500
33/33 [=====] - 0s 9ms/step - loss: 0.0076 - mae: 0.0941 - val_loss: 0.0055 - val_mae: 0.0811
Epoch 8/500
33/33 [=====] - 0s 8ms/step - loss: 0.0072 - mae: 0.0900 - val_loss: 0.0050 - val_mae: 0.0779
Epoch 9/500
33/33 [=====] - 0s 8ms/step - loss: 0.0068 - mae: 0.0869 - val_loss: 0.0046 - val_mae: 0.0751
Epoch 10/500
33/33 [=====] - 0s 8ms/step - loss: 0.0066 - mae: 0.0844 - val_loss: 0.0045 - val_mae: 0.0737
Epoch 11/500
33/33 [=====] - 0s 9ms/step - loss: 0.0063 - mae: 0.0822 - val_loss: 0.0041 - val_mae: 0.0709
Epoch 12/500
33/33 [=====] - 0s 9ms/step - loss: 0.0061 - mae: 0.0804 - val_loss: 0.0039 - val_mae: 0.0688
Epoch 13/500
33/33 [=====] - 0s 8ms/step - loss: 0.0060 - mae: 0.0796 - val_loss: 0.0039 - val_mae: 0.0690
Epoch 14/500
33/33 [=====] - 0s 8ms/step - loss: 0.0059 - mae: 0.0777 - val_loss: 0.0036 - val_mae: 0.0656
Epoch 15/500
33/33 [=====] - 0s 8ms/step - loss: 0.0058 - mae: 0.0766 - val_loss: 0.0035 - val_mae: 0.0649
Epoch 16/500
33/33 [=====] - 0s 8ms/step - loss: 0.0056 - mae: 0.0755 - val_loss: 0.0034 - val_mae: 0.0638
Epoch 17/500
33/33 [=====] - 0s 8ms/step - loss: 0.0055 - mae: 0.0744 - val_loss: 0.0033 - val_mae: 0.0633
Epoch 18/500
<<303 more lines>>
Epoch 170/500
33/33 [=====] - 0s 7ms/step - loss: 0.0032 - mae: 0.0474 - val_loss: 0.0014 - val_mae: 0.0359
Epoch 171/500
33/33 [=====] - 0s 9ms/step - loss: 0.0032 - mae: 0.0477 - val_loss: 0.0014 - val_mae: 0.0359
Epoch 172/500
33/33 [=====] - 0s 8ms/step - loss: 0.0032 - mae: 0.0479 - val_loss: 0.0014 - val_mae: 0.0353
Epoch 173/500
33/33 [=====] - 0s 8ms/step - loss: 0.0032 - mae: 0.0480 - val_loss: 0.0014 - val_mae: 0.0359
Epoch 174/500
33/33 [=====] - 0s 9ms/step - loss: 0.0032 - mae: 0.0481 - val_loss: 0.0015 - val_mae: 0.0365
Epoch 175/500
33/33 [=====] - 0s 8ms/step - loss: 0.0032 - mae: 0.0476 - val_loss: 0.0014 - val_mae: 0.0358
Epoch 176/500
33/33 [=====] - 0s 8ms/step - loss: 0.0032 - mae: 0.0474 - val_loss: 0.0014 - val_mae: 0.0355
Epoch 177/500
33/33 [=====] - 0s 8ms/step - loss: 0.0032 - mae: 0.0480 - val_loss: 0.0014 - val_mae: 0.0362
Epoch 178/500
33/33 [=====] - 0s 9ms/step - loss: 0.0032 - mae: 0.0476 - val_loss: 0.0014 - val_mae: 0.0353
Epoch 179/500
33/33 [=====] - 0s 8ms/step - loss: 0.0032 - mae: 0.0481 - val_loss: 0.0014 - val_mae: 0.0357
Epoch 180/500
33/33 [=====] - 0s 9ms/step - loss: 0.0032 - mae: 0.0476 - val_loss: 0.0014 - val_mae: 0.0352

```

```

X = mulvar_valid.to_numpy()[np.newaxis, :seq_length] # shape [1, 56, 5]
Y_pred = ahead_model.predict(X) # shape [1, 14]

```

Bây giờ hãy tạo một RNN dự đoán 14 bước tiếp theo tại mỗi bước thời gian. Nghĩa là, thay vì chỉ dự báo các bước thời gian từ 56 đến 69 dựa trên các bước thời gian từ 0 đến 55, nó sẽ dự báo các bước thời gian từ 1 đến 14 tại bước thời gian 0, sau đó là các bước từ 2 đến 15 tại bước thời gian 1, v.v., và cuối cùng nó sẽ dự báo các bước từ 56 đến 69 tại bước thời gian cuối cùng. Lưu ý rằng mô hình này mang tính nhân quả (causal): khi đưa ra dự đoán tại bất kỳ bước thời gian nào, nó chỉ có thể nhìn thấy các bước thời gian trong quá khứ.

Để chuẩn bị các tập dữ liệu, chúng ta có thể sử dụng `to_windows()` hai lần để có được các chuỗi cửa sổ liên tiếp, như thế này:

```
my_series = tf.data.Dataset.range(7)
dataset = to_windows(to_windows(my_series, 3), 4)
list(dataset)
```

```
[<tf.Tensor: shape=(4, 3), dtype=int64, numpy=
array([[0, 1, 2],
       [1, 2, 3],
       [2, 3, 4],
       [3, 4, 5]])>,
 <tf.Tensor: shape=(4, 3), dtype=int64, numpy=
array([[1, 2, 3],
       [2, 3, 4],
       [3, 4, 5],
       [4, 5, 6]])>]
```

Sau đó, chúng ta có thể chia các phần tử này thành các đầu vào (inputs) và mục tiêu (targets) mong muốn:

```
dataset = dataset.map(lambda S: (S[:, 0], S[:, 1:]))
list(dataset)
```

```
[(<tf.Tensor: shape=(4,), dtype=int64, numpy=array([0, 1, 2, 3])>,
 <tf.Tensor: shape=(4, 2), dtype=int64, numpy=
array([[1, 2],
       [2, 3],
       [3, 4],
       [4, 5]])>),
 (<tf.Tensor: shape=(4,), dtype=int64, numpy=array([1, 2, 3, 4])>,
 <tf.Tensor: shape=(4, 2), dtype=int64, numpy=
array([[2, 3],
       [3, 4],
       [4, 5],
       [5, 6]])>)]
```

Hãy đóng gói ý tưởng này vào một hàm tiện ích. Nó cũng sẽ xử lý việc xáo trộn (tùy chọn) và chia lô (batching):

```
def to_seq2seq_dataset(series, seq_length=56, ahead=14, target_col=1,
                       batch_size=32, shuffle=False, seed=None):
    ds = to_windows(tf.data.Dataset.from_tensor_slices(series), ahead + 1)
    ds = to_windows(ds, seq_length).map(
        lambda S: (S[:, 0], S[:, 1:, target_col]))
    if shuffle:
        ds = ds.shuffle(8 * batch_size, seed=seed)
    return ds.batch(batch_size)
```

```
seq2seq_train = to_seq2seq_dataset(mulvar_train, shuffle=True, seed=42)
seq2seq_valid = to_seq2seq_dataset(mulvar_valid)
```

```
tf.random.set_seed(42) # extra code - ensures reproducibility
seq2seq_model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(32, return_sequences=True, input_shape=[None, 5]),
    tf.keras.layers.Dense(14)
    # equivalent: tf.keras.layers.TimeDistributed(tf.keras.layers.Dense(14))
    # also equivalent: tf.keras.layers.Conv1D(14, kernel_size=1)
])
```

```
fit_and_evaluate(seq2seq_model, seq2seq_train, seq2seq_valid,
                 learning_rate=0.1)
```

```
Epoch 1/500
33/33 [=====] - 1s 17ms/step - loss: 0.0754 - mae: 0.2785 - val_loss: 0.0163 - val_mae: 0.1379
Epoch 2/500
33/33 [=====] - 0s 11ms/step - loss: 0.0097 - mae: 0.1050 - val_loss: 0.0071 - val_mae: 0.0853
Epoch 3/500
33/33 [=====] - 0s 11ms/step - loss: 0.0069 - mae: 0.0846 - val_loss: 0.0063 - val_mae: 0.0790
Epoch 4/500
```

```

33/33 [=====] - 0s 10ms/step - loss: 0.0060 - mae: 0.0773 - val_loss: 0.0056 - val_mae: 0.0729
Epoch 5/500
33/33 [=====] - 0s 10ms/step - loss: 0.0055 - mae: 0.0722 - val_loss: 0.0049 - val_mae: 0.0662
Epoch 6/500
33/33 [=====] - 0s 10ms/step - loss: 0.0052 - mae: 0.0690 - val_loss: 0.0051 - val_mae: 0.0683
Epoch 7/500
33/33 [=====] - 0s 11ms/step - loss: 0.0049 - mae: 0.0663 - val_loss: 0.0046 - val_mae: 0.0626
Epoch 8/500
33/33 [=====] - 0s 11ms/step - loss: 0.0047 - mae: 0.0640 - val_loss: 0.0043 - val_mae: 0.0589
Epoch 9/500
33/33 [=====] - 0s 11ms/step - loss: 0.0046 - mae: 0.0627 - val_loss: 0.0041 - val_mae: 0.0560
Epoch 10/500
33/33 [=====] - 0s 11ms/step - loss: 0.0045 - mae: 0.0616 - val_loss: 0.0043 - val_mae: 0.0589
Epoch 11/500
33/33 [=====] - 0s 11ms/step - loss: 0.0044 - mae: 0.0608 - val_loss: 0.0042 - val_mae: 0.0580
Epoch 12/500
33/33 [=====] - 0s 10ms/step - loss: 0.0043 - mae: 0.0594 - val_loss: 0.0040 - val_mae: 0.0554
Epoch 13/500
33/33 [=====] - 0s 11ms/step - loss: 0.0042 - mae: 0.0584 - val_loss: 0.0041 - val_mae: 0.0572
Epoch 14/500
33/33 [=====] - 0s 11ms/step - loss: 0.0042 - mae: 0.0577 - val_loss: 0.0042 - val_mae: 0.0580
Epoch 15/500
33/33 [=====] - 0s 11ms/step - loss: 0.0042 - mae: 0.0579 - val_loss: 0.0038 - val_mae: 0.0530
Epoch 16/500
33/33 [=====] - 0s 10ms/step - loss: 0.0041 - mae: 0.0573 - val_loss: 0.0039 - val_mae: 0.0534
Epoch 17/500
33/33 [=====] - 0s 11ms/step - loss: 0.0041 - mae: 0.0566 - val_loss: 0.0038 - val_mae: 0.0530
Epoch 18/500
<<219 more lines>>
Epoch 128/500
33/33 [=====] - 0s 10ms/step - loss: 0.0033 - mae: 0.0484 - val_loss: 0.0036 - val_mae: 0.0470
Epoch 129/500
33/33 [=====] - 0s 10ms/step - loss: 0.0033 - mae: 0.0489 - val_loss: 0.0036 - val_mae: 0.0472
Epoch 130/500
33/33 [=====] - 0s 11ms/step - loss: 0.0032 - mae: 0.0476 - val_loss: 0.0036 - val_mae: 0.0473
Epoch 131/500
33/33 [=====] - 0s 10ms/step - loss: 0.0032 - mae: 0.0483 - val_loss: 0.0036 - val_mae: 0.0479
Epoch 132/500
33/33 [=====] - 0s 10ms/step - loss: 0.0033 - mae: 0.0492 - val_loss: 0.0037 - val_mae: 0.0489
Epoch 133/500
33/33 [=====] - 0s 10ms/step - loss: 0.0033 - mae: 0.0499 - val_loss: 0.0036 - val_mae: 0.0480
Epoch 134/500
33/33 [=====] - 0s 11ms/step - loss: 0.0033 - mae: 0.0486 - val_loss: 0.0035 - val_mae: 0.0469
Epoch 135/500
33/33 [=====] - 0s 10ms/step - loss: 0.0033 - mae: 0.0486 - val_loss: 0.0035 - val_mae: 0.0468
Epoch 136/500
33/33 [=====] - 0s 10ms/step - loss: 0.0033 - mae: 0.0491 - val_loss: 0.0035 - val_mae: 0.0467
Epoch 137/500
33/33 [=====] - 0s 10ms/step - loss: 0.0033 - mae: 0.0493 - val_loss: 0.0035 - val_mae: 0.0471
Epoch 138/500
33/33 [=====] - 0s 10ms/step - loss: 0.0033 - mae: 0.0486 - val_loss: 0.0036 - val_mae: 0.0476

```

```

X = mulvar_valid.to_numpy()[np.newaxis, :seq_length]
y_pred_14 = seq2seq_model.predict(X)[0, -1] # only the last time step's output

```

```

Y_pred_valid = seq2seq_model.predict(seq2seq_valid)
for ahead in range(14):
    preds = pd.Series(Y_pred_valid[: -1, -1, ahead],
                      index=mulvar_valid.index[56 + ahead : -14 + ahead])
    mae = (preds - mulvar_valid["rail"]).abs().mean() * 1e6
    print(f"MAE for +{ahead + 1}: {mae:,.0f}")

```

```

MAE for +1: 25,519
MAE for +2: 26,274
MAE for +3: 27,054
MAE for +4: 29,324
MAE for +5: 28,992
MAE for +6: 31,739
MAE for +7: 32,847
MAE for +8: 33,282
MAE for +9: 33,072
MAE for +10: 29,752
MAE for +11: 37,468
MAE for +12: 35,125
MAE for +13: 34,614
MAE for +14: 34,322

```

## ✦ Deep RNNs với Layer Norm



```

class LNSimpleRNNCell(tf.keras.layers.Layer):
    def __init__(self, units, activation="tanh", **kwargs):
        super().__init__(**kwargs)
        self.state_size = units
        self.output_size = units
        self.simple_rnn_cell = tf.keras.layers.SimpleRNNCell(units,
                                                                activation=None)

        self.layer_norm = tf.keras.layers.LayerNormalization()
        self.activation = tf.keras.activations.get(activation)

    def call(self, inputs, states):
        outputs, new_states = self.simple_rnn_cell(inputs, states)
        norm_outputs = self.activation(self.layer_norm(outputs))
        return norm_outputs, [norm_outputs]

```

```

tf.random.set_seed(42) # extra code - ensures reproducibility
custom_ln_model = tf.keras.Sequential([
    tf.keras.layers.RNN(LNSimpleRNNCell(32), return_sequences=True,
                        input_shape=[None, 5]),
    tf.keras.layers.Dense(14)
])

```

Just training for 5 epochs to show that it works (you can increase this if you want):

```

fit_and_evaluate(custom_ln_model, seq2seq_train, seq2seq_valid,
                 learning_rate=0.1, epochs=5)

Epoch 1/5
33/33 [=====] - 2s 25ms/step - loss: 0.0809 - mae: 0.2898 - val_loss: 0.0178 - val_mae: 0.1511
Epoch 2/5
33/33 [=====] - 1s 18ms/step - loss: 0.0149 - mae: 0.1438 - val_loss: 0.0156 - val_mae: 0.1245
Epoch 3/5
33/33 [=====] - 1s 18ms/step - loss: 0.0120 - mae: 0.1281 - val_loss: 0.0131 - val_mae: 0.1160
Epoch 4/5
33/33 [=====] - 1s 17ms/step - loss: 0.0105 - mae: 0.1167 - val_loss: 0.0118 - val_mae: 0.1095
Epoch 5/5
33/33 [=====] - 1s 17ms/step - loss: 0.0093 - mae: 0.1067 - val_loss: 0.0105 - val_mae: 0.1038
3/3 [=====] - 0s 14ms/step - loss: 0.0105 - mae: 0.1038
103751.08569860458

```

## ✧ Tài liệu bổ sung – Tạo một lớp RNN tùy chỉnh

Lớp RNN không hề ma thuật. Trên thực tế, việc triển khai lớp RNN của riêng bạn không quá khó:

```

class MyRNN(tf.keras.layers.Layer):
    def __init__(self, cell, return_sequences=False, **kwargs):
        super().__init__(**kwargs)
        self.cell = cell
        self.return_sequences = return_sequences

    def get_initial_state(self, inputs):
        try:
            return self.cell.get_initial_state(inputs)
        except AttributeError:
            # fallback to zeros if self.cell has no get_initial_state() method
            batch_size = tf.shape(inputs)[0]
            return [tf.zeros([batch_size, self.cell.state_size],
                             dtype=inputs.dtype)]

    @tf.function
    def call(self, inputs):
        states = self.get_initial_state(inputs)
        shape = tf.shape(inputs)
        batch_size = shape[0]
        n_steps = shape[1]
        sequences = tf.TensorArray(
            inputs.dtype, size=(n_steps if self.return_sequences else 0))
        outputs = tf.zeros(shape=[batch_size, self.cell.output_size],
                           dtype=inputs.dtype)
        for step in tf.range(n_steps):
            outputs, states = self.cell(inputs[:, step], states)
            if self.return_sequences:

```

```

        sequences = sequences.write(step, outputs)

    if self.return_sequences:
        # stack the outputs into an array of shape
        # [time steps, batch size, dims], then transpose it to shape
        # [batch size, time steps, dims]
        return tf.transpose(sequences.stack(), [1, 0, 2])
    else:
        return outputs

```

Lưu ý rằng `@tf.function` yêu cầu biến `outputs` phải được tạo trước vòng lặp `for`. Tương tự, nó yêu cầu biến `sequences` phải được tạo trước câu lệnh `if` nơi nó được sử dụng.

```

tf.random.set_seed(42)

custom_model = tf.keras.Sequential([
    MyRNN(LNSimpleRNNCell(32), return_sequences=True, input_shape=[None, 5]),
    tf.keras.layers.Dense(14)
])

```

Just training for 5 epochs to show that it works (you can increase this if you want):

```

fit_and_evaluate(custom_model, seq2seq_train, seq2seq_valid,
                  learning_rate=0.1, epochs=5)

Epoch 1/5
33/33 [=====] - 2s 26ms/step - loss: 0.0814 - mae: 0.2916 - val_loss: 0.0176 - val_mae: 0.1544
Epoch 2/5
33/33 [=====] - 1s 20ms/step - loss: 0.0151 - mae: 0.1440 - val_loss: 0.0157 - val_mae: 0.1247
Epoch 3/5
33/33 [=====] - 1s 19ms/step - loss: 0.0119 - mae: 0.1281 - val_loss: 0.0134 - val_mae: 0.1160
Epoch 4/5
33/33 [=====] - 1s 18ms/step - loss: 0.0105 - mae: 0.1162 - val_loss: 0.0111 - val_mae: 0.1084
Epoch 5/5
33/33 [=====] - 1s 18ms/step - loss: 0.0093 - mae: 0.1068 - val_loss: 0.0103 - val_mae: 0.1029
3/3 [=====] - 0s 14ms/step - loss: 0.0103 - mae: 0.1029
102874.92722272873

```

## ✦ LSTMs

```

tf.random.set_seed(42) # extra code - ensures reproducibility
lstm_model = tf.keras.models.Sequential([
    tf.keras.layers.LSTM(32, return_sequences=True, input_shape=[None, 5]),
    tf.keras.layers.Dense(14)
])

```

Just training for 5 epochs to show that it works (you can increase this if you want):

```

fit_and_evaluate(lstm_model, seq2seq_train, seq2seq_valid,
                  learning_rate=0.1, epochs=5)

Epoch 1/5
33/33 [=====] - 2s 29ms/step - loss: 0.0535 - mae: 0.2517 - val_loss: 0.0187 - val_mae: 0.1716
Epoch 2/5
33/33 [=====] - 1s 16ms/step - loss: 0.0176 - mae: 0.1598 - val_loss: 0.0176 - val_mae: 0.1473
Epoch 3/5
33/33 [=====] - 1s 16ms/step - loss: 0.0160 - mae: 0.1528 - val_loss: 0.0168 - val_mae: 0.1433
Epoch 4/5
33/33 [=====] - 1s 16ms/step - loss: 0.0152 - mae: 0.1485 - val_loss: 0.0161 - val_mae: 0.1388
Epoch 5/5
33/33 [=====] - 1s 16ms/step - loss: 0.0145 - mae: 0.1443 - val_loss: 0.0154 - val_mae: 0.1352
3/3 [=====] - 0s 14ms/step - loss: 0.0154 - mae: 0.1352
135186.25497817993

```

## ✦ GRUs

```

tf.random.set_seed(42) # extra code - ensures reproducibility
gru_model = tf.keras.Sequential([
    tf.keras.layers.GRU(32, return_sequences=True, input_shape=[None, 5]),

```

```
tf.keras.layers.Dense(14)
])
```

Just training for 5 epochs to show that it works (you can increase this if you want):

```
fit_and_evaluate(gru_model, seq2seq_train, seq2seq_valid,
                  learning_rate=0.1, epochs=5)
```

```
Epoch 1/5
33/33 [=====] - 2s 29ms/step - loss: 0.0516 - mae: 0.2489 - val_loss: 0.0165 - val_mae: 0.1529
Epoch 2/5
33/33 [=====] - 1s 18ms/step - loss: 0.0145 - mae: 0.1386 - val_loss: 0.0139 - val_mae: 0.1260
Epoch 3/5
33/33 [=====] - 1s 18ms/step - loss: 0.0118 - mae: 0.1249 - val_loss: 0.0121 - val_mae: 0.1170
Epoch 4/5
33/33 [=====] - 1s 18ms/step - loss: 0.0106 - mae: 0.1166 - val_loss: 0.0111 - val_mae: 0.1109
Epoch 5/5
33/33 [=====] - 1s 18ms/step - loss: 0.0098 - mae: 0.1107 - val_loss: 0.0104 - val_mae: 0.1071
3/3 [=====] - 0s 14ms/step - loss: 0.0104 - mae: 0.1071
107093.29694509506
```

## ✧ Sử dụng các lớp tích chập một chiều (1D Conv) để xử lý chuỗi

```
|-----0-----|      |-----3-----|      |-----52-----|
      |-----1-----|      |-----4-----|      ...      |-----53-----|
              |-----2-----|      |-----5-----51-----|      |-----54-----|
X:  0  1  2  3  4  5  6  7  8  9 10 11 12 ... 104 105 106 107 108 109 110 111
Y:      from 4      6      8      10      12      ...      106      108      110      112
      to 17      19      21      23      25      ...      119      121      123      125
```

```
tf.random.set_seed(42) # extra code - ensures reproducibility
conv_rnn_model = tf.keras.Sequential([
    tf.keras.layers.Conv1D(filters=32, kernel_size=4, strides=2,
                           activation="relu", input_shape=[None, 5]),
    tf.keras.layers.GRU(32, return_sequences=True),
    tf.keras.layers.Dense(14)
])

longer_train = to_seq2seq_dataset(mulvar_train, seq_length=112,
                                  shuffle=True, seed=42)
longer_valid = to_seq2seq_dataset(mulvar_valid, seq_length=112)
downsampled_train = longer_train.map(lambda X, Y: (X, Y[:, 3::2]))
downsampled_valid = longer_valid.map(lambda X, Y: (X, Y[:, 3::2]))
```

Just training for 5 epochs to show that it works (you can increase this if you want):

```
fit_and_evaluate(conv_rnn_model, downsampled_train, downsampled_valid,
                  learning_rate=0.1, epochs=5)
```

```
Epoch 1/5
31/31 [=====] - 2s 30ms/step - loss: 0.0482 - mae: 0.2420 - val_loss: 0.0214 - val_mae: 0.1616
Epoch 2/5
31/31 [=====] - 1s 18ms/step - loss: 0.0165 - mae: 0.1532 - val_loss: 0.0171 - val_mae: 0.1423
Epoch 3/5
31/31 [=====] - 1s 18ms/step - loss: 0.0144 - mae: 0.1447 - val_loss: 0.0157 - val_mae: 0.1342
Epoch 4/5
31/31 [=====] - 1s 17ms/step - loss: 0.0130 - mae: 0.1361 - val_loss: 0.0141 - val_mae: 0.1254
Epoch 5/5
31/31 [=====] - 1s 17ms/step - loss: 0.0115 - mae: 0.1256 - val_loss: 0.0124 - val_mae: 0.1159
1/1 [=====] - 0s 88ms/step - loss: 0.0124 - mae: 0.1159
115850.42625665665
```

## ✧ WaveNet

```
⋮
C2  /\ /\ /\ /\ /\ /\ /\ /\ /\ /\ /\ ...
    \ / \ / \ / \ / \ / \ / \ / \ / \ / \
```

```

      / \      / \      / \
C1  /\ /\ /\ /\ /\ /\ /\ /\ /\ /\ /\ /\ /\ /\ ... \
X: 0  1  2  3  4  5  6  7  8  9 10 11 12 ... 111
Y: 1  2  3  4  5  6  7  8  9 10 11 12 13 ... 112
    /14 15 16 17 18 19 20 21 22  23 24 25 26 ... 125

```

```

tf.random.set_seed(42) # extra code - ensures reproducibility
wavenet_model = tf.keras.Sequential()
wavenet_model.add(tf.keras.layers.InputLayer(input_shape=[None, 5]))
for rate in (1, 2, 4, 8) * 2:
    wavenet_model.add(tf.keras.layers.Conv1D(
        filters=32, kernel_size=2, padding="causal", activation="relu",
        dilation_rate=rate))
wavenet_model.add(tf.keras.layers.Conv1D(filters=14, kernel_size=1))

```

Just training for 5 epochs to show that it works (you can increase this if you want):

```

fit_and_evaluate(wavenet_model, longer_train, longer_valid,
                 learning_rate=0.1, epochs=5)

```

```

Epoch 1/5
31/31 [=====] - 2s 26ms/step - loss: 0.0796 - mae: 0.3159 - val_loss: 0.0239 - val_mae: 0.1723
Epoch 2/5
31/31 [=====] - 1s 16ms/step - loss: 0.0172 - mae: 0.1585 - val_loss: 0.0182 - val_mae: 0.1545
Epoch 3/5
31/31 [=====] - 1s 16ms/step - loss: 0.0159 - mae: 0.1561 - val_loss: 0.0181 - val_mae: 0.1505
Epoch 4/5
31/31 [=====] - 1s 16ms/step - loss: 0.0155 - mae: 0.1535 - val_loss: 0.0175 - val_mae: 0.1479
Epoch 5/5
31/31 [=====] - 1s 17ms/step - loss: 0.0147 - mae: 0.1488 - val_loss: 0.0166 - val_mae: 0.1407
1/1 [=====] - 0s 74ms/step - loss: 0.0166 - mae: 0.1407
140713.95993232727

```

## ✧ Tài liệu bổ sung – Triển khai Wavenet

Đây là mô hình WaveNet nguyên bản được định nghĩa trong bài báo: nó sử dụng các Gated Activation Units thay vì ReLU và các kết nối skip được tham số hóa.

```

class GatedActivationUnit(tf.keras.layers.Layer):
    def __init__(self, activation="tanh", **kwargs):
        super().__init__(**kwargs)
        self.activation = tf.keras.activations.get(activation)

    def call(self, inputs):
        n_filters = inputs.shape[-1] // 2
        linear_output = self.activation(inputs[..., :n_filters])
        gate = tf.keras.activations.sigmoid(inputs[..., n_filters:])
        return self.activation(linear_output) * gate

```

```

def wavenet_residual_block(inputs, n_filters, dilation_rate):
    z = tf.keras.layers.Conv1D(2 * n_filters, kernel_size=2, padding="causal",
                               dilation_rate=dilation_rate)(inputs)
    z = GatedActivationUnit()(z)
    z = tf.keras.layers.Conv1D(n_filters, kernel_size=1)(z)
    return tf.keras.layers.Add()([z, inputs]), z

```

```

tf.random.set_seed(42)

n_layers_per_block = 3 # 10 in the paper
n_blocks = 1 # 3 in the paper
n_filters = 32 # 128 in the paper
n_outputs = 14 # 256 in the paper

inputs = tf.keras.layers.Input(shape=[None, 5])
z = tf.keras.layers.Conv1D(n_filters, kernel_size=2, padding="causal")(inputs)
skip_to_last = []
for dilation_rate in [2**i for i in range(n_layers_per_block)] * n_blocks:
    z, skip = wavenet_residual_block(z, n_filters, dilation_rate)
    skip_to_last.append(skip)

```

```

z = tf.keras.activations.relu(tf.keras.layers.Add()(skip_to_last))
z = tf.keras.layers.Conv1D(n_filters, kernel_size=1, activation="relu")(z)
Y_preds = tf.keras.layers.Conv1D(n_outputs, kernel_size=1)(z)

full_wavenet_model = tf.keras.Model(inputs=[inputs], outputs=[Y_preds])

```

Just training for 5 epochs to show that it works (you can increase this if you want):

```

fit_and_evaluate(full_wavenet_model, longer_train, longer_valid,
                 learning_rate=0.1, epochs=5)

```

```

Epoch 1/5
31/31 [=====] - 2s 26ms/step - loss: 0.0706 - mae: 0.2861 - val_loss: 0.0209 - val_mae: 0.1630
Epoch 2/5
31/31 [=====] - 1s 18ms/step - loss: 0.0137 - mae: 0.1398 - val_loss: 0.0140 - val_mae: 0.1273
Epoch 3/5
31/31 [=====] - 1s 20ms/step - loss: 0.0104 - mae: 0.1190 - val_loss: 0.0116 - val_mae: 0.1125
Epoch 4/5
31/31 [=====] - 1s 18ms/step - loss: 0.0086 - mae: 0.1048 - val_loss: 0.0096 - val_mae: 0.1020
Epoch 5/5
31/31 [=====] - 1s 19ms/step - loss: 0.0073 - mae: 0.0942 - val_loss: 0.0087 - val_mae: 0.0953
1/1 [=====] - 0s 71ms/step - loss: 0.0087 - mae: 0.0953
95349.08086061478

```

Trong chương này, chúng ta đã khám phá các nguyên tắc cơ bản của RNN và sử dụng chúng để xử lý các chuỗi (cụ thể là chuỗi thời gian). Trong chương tiếp theo, chúng ta sẽ sử dụng RNN cho Xử lý ngôn ngữ tự nhiên (NLP) và tìm hiểu về Transformer.

## ✧ Lời giải bài tập

### ✧ 1. to 8.

1. Here are a few RNN applications:

- For a sequence-to-sequence RNN: predicting the weather (or any other time series), machine translation (using an Encoder–Decoder architecture), video captioning, speech to text, music generation (or other sequence generation), identifying the chords of a song
- For a sequence-to-vector RNN: classifying music samples by music genre, analyzing the sentiment of a book review, predicting what word an aphasic patient is thinking of based on readings from brain implants, predicting the probability that a user will want to watch a movie based on their watch history (this is one of many possible implementations of *collaborative filtering* for a recommender system)
- For a vector-to-sequence RNN: image captioning, creating a music playlist based on an embedding of the current artist, generating a melody based on a set of parameters, locating pedestrians in a picture (e.g., a video frame from a self-driving car's camera)

2. An RNN layer must have three-dimensional inputs: the first dimension is the batch dimension (its size is the batch size), the second dimension represents the time (its size is the number of time steps), and the third dimension holds the inputs at each time step (its size is the number of input features per time step). For example, if you want to process a batch containing 5 time series of 10 time steps each, with 2 values per time step (e.g., the temperature and the wind speed), the shape will be [5, 10, 2]. The outputs are also three-dimensional, with the same first two dimensions, but the last dimension is equal to the number of neurons. For example, if an RNN layer with 32 neurons processes the batch we just discussed, the output will have a shape of [5, 10, 32].

3. To build a deep sequence-to-sequence RNN using Keras, you must set `return_sequences=True` for all RNN layers. To build a sequence-to-vector RNN, you must set `return_sequences=True` for all layers except for the top RNN layer, which must have `return_sequences=False` (or do not set this argument at all, since `False` is the default).

4. If you have a daily univariate time series, and you want to forecast the next seven days, the simplest RNN architecture you can use is a stack of RNN layers (all with `return_sequences=True` except for the top RNN layer), using seven neurons in the output RNN layer. You can then train this model using random windows from the time series (e.g., sequences of 30 consecutive days as the inputs, and a vector containing the values of the next 7 days as the target). This is a sequence-to-vector RNN. Alternatively, you could set `return_sequences=True` for all RNN layers to create a sequence-to-sequence RNN. You can train this model using random windows from the time series, with sequences of the same length as the inputs as the targets. Each target sequence should have seven values per time step (e.g., for time step  $t$ , the target should be a vector containing the values at time steps  $t + 1$  to  $t + 7$ ).

5. The two main difficulties when training RNNs are unstable gradients (exploding or vanishing) and a very limited short-term memory. These problems both get worse when dealing with long sequences. To alleviate the unstable gradients problem, you can use a smaller learning rate, use a saturating activation function such as the hyperbolic tangent (which is the default), and possibly use gradient clipping, Layer Normalization, or dropout at each time step. To tackle the limited short-term memory problem, you can use `LSTM` or `GRU` layers (this also helps with the unstable gradients problem).
6. An LSTM cell's architecture looks complicated, but it's actually not too hard if you understand the underlying logic. The cell has a short-term state vector and a long-term state vector. At each time step, the inputs and the previous short-term state are fed to a simple RNN cell and three gates: the forget gate decides what to remove from the long-term state, the input gate decides which part of the output of the simple RNN cell should be added to the long-term state, and the output gate decides which part of the long-term state should be output at this time step (after going through the tanh activation function). The new short-term state is equal to the output of the cell. See Figure 15–12.
7. An RNN layer is fundamentally sequential: in order to compute the outputs at time step  $t$ , it has to first compute the outputs at all earlier time steps. This makes it impossible to parallelize. On the other hand, a 1D convolutional layer lends itself well to parallelization since it does not hold a state between time steps. In other words, it has no memory: the output at any time step can be computed based only on a small window of values from the inputs without having to know all the past values. Moreover, since a 1D convolutional layer is not recurrent, it suffers less from unstable gradients. One or more 1D convolutional layers can be useful in an RNN to efficiently preprocess the inputs, for example to reduce their temporal resolution (downsampling) and thereby help the RNN layers detect long-term patterns. In fact, it is possible to use only convolutional layers, for example by building a WaveNet architecture.
8. To classify videos based on their visual content, one possible architecture could be to take (say) one frame per second, then run every frame through the same convolutional neural network (e.g., a pretrained Xception model, possibly frozen if your dataset is not large), feed the sequence of outputs from the CNN to a sequence-to-vector RNN, and finally run its output through a softmax layer, giving you all the class probabilities. For training you would use cross entropy as the cost function. If you wanted to use the audio for classification as well, you could use a stack of strided 1D convolutional layers to reduce the temporal resolution from thousands of audio frames per second to just one per second (to match the number of images per second), and concatenate the output sequence to the inputs of the sequence-to-vector RNN (along the last dimension).

## ✓ 9. Xử lý tập dữ liệu SketchRNN

*Bài tập: Huấn luyện một mô hình phân loại cho tập dữ liệu SketchRNN, có sẵn trong TensorFlow Datasets.*

Dữ liệu hiện có sẵn dưới dạng TFRecords, hãy tải xuống (có thể mất một lúc vì nó nặng khoảng 1 GB):

```
tf_download_root = "http://download.tensorflow.org/data/"
filename = "quickdraw_tutorial_dataset_v1.tar.gz"
filepath = tf.keras.utils.get_file(filename,
                                    tf_download_root + filename,
                                    cache_dir=".",
                                    extract=True)
```

```
Downloading data from http://download.tensorflow.org/data/quickdraw\_tutorial\_dataset\_v1.tar.gz
1065304064/1065301781 [=====] - 230s 0us/step
1065312256/1065301781 [=====] - 230s 0us/step
```

```
if "_extracted" in filepath:
    quickdraw_dir = Path(filepath)
else:
    quickdraw_dir = Path(filepath).parent
train_files = sorted(
    [str(path) for path in quickdraw_dir.glob("training.tfrecord-*")]
)
eval_files = sorted(
    [str(path) for path in quickdraw_dir.glob("eval.tfrecord-*")]
)
```

```
train_files
```

```
['datasets/training.tfrecord-00000-of-00010',
 'datasets/training.tfrecord-00001-of-00010',
 'datasets/training.tfrecord-00002-of-00010',
 'datasets/training.tfrecord-00003-of-00010',
 'datasets/training.tfrecord-00004-of-00010',
 'datasets/training.tfrecord-00005-of-00010',
 'datasets/training.tfrecord-00006-of-00010',
```

```
'datasets/training.tfrecord-00007-of-00010',  
'datasets/training.tfrecord-00008-of-00010',  
'datasets/training.tfrecord-00009-of-00010']
```

eval\_files

```
['datasets/eval.tfrecord-00000-of-00010',  
'datasets/eval.tfrecord-00001-of-00010',  
'datasets/eval.tfrecord-00002-of-00010',  
'datasets/eval.tfrecord-00003-of-00010',  
'datasets/eval.tfrecord-00004-of-00010',  
'datasets/eval.tfrecord-00005-of-00010',  
'datasets/eval.tfrecord-00006-of-00010',  
'datasets/eval.tfrecord-00007-of-00010',  
'datasets/eval.tfrecord-00008-of-00010',  
'datasets/eval.tfrecord-00009-of-00010']
```

```
with open(quickdraw_dir / "eval.tfrecord.classes") as test_classes_file:  
    test_classes = test_classes_file.readlines()  
  
with open(quickdraw_dir / "training.tfrecord.classes") as train_classes_file:  
    train_classes = train_classes_file.readlines()
```

```
assert train_classes == test_classes  
class_names = [name.strip().lower() for name in train_classes]
```

sorted(class\_names)

```
['aircraft carrier',  
'airplane',  
'alarm clock',  
'ambulance',  
'angel',  
'animal migration',  
'ant',  
'anvil',  
'apple',  
'arm',  
'asparagus',  
'axe',  
'backpack',  
'banana',  
'bandage',  
'barn',  
'baseball',  
'baseball bat',  
'basket',  
'basketball',  
'bat',  
'bathtub',  
'beach',  
'bear',  
'beard',  
'bed',  
'bee',  
'belt',  
'bench',  
'bicycle',  
'binoculars',  
'bird',  
'birthday cake',  
'blackberry',  
'blueberry',  
'book',  
'boomerang',  
'bottlecap',  
'bowtie',  
'bracelet',  
'brain',  
'bread',  
'bridge',  
'broccoli',  
'broom',  
'bucket',  
'bulldozer',  
'bus',  
'bush',  
'butterfly',  
'cactus',  
'cake',
```

```
'calculator',
'calendar',
'camel',
'camera',
'camouflage',
```

```
def parse(data_batch):
    feature_descriptions = {
        "ink": tf.io.VarLenFeature(dtype=tf.float32),
        "shape": tf.io.FixedLenFeature([2], dtype=tf.int64),
        "class_index": tf.io.FixedLenFeature([1], dtype=tf.int64)
    }
    examples = tf.io.parse_example(data_batch, feature_descriptions)
    flat_sketches = tf.sparse.to_dense(examples["ink"])
    sketches = tf.reshape(flat_sketches, shape=[tf.size(data_batch), -1, 3])
    lengths = examples["shape"][:, 0]
    labels = examples["class_index"][:, 0]
    return sketches, lengths, labels
```

```
def quickdraw_dataset(filepaths, batch_size=32, shuffle_buffer_size=None,
                      n_parse_threads=5, n_read_threads=5, cache=False):
    dataset = tf.data.TFRecordDataset(filepaths,
                                     num_parallel_reads=n_read_threads)
    if cache:
        dataset = dataset.cache()
    if shuffle_buffer_size:
        dataset = dataset.shuffle(shuffle_buffer_size)
    dataset = dataset.batch(batch_size)
    dataset = dataset.map(parse, num_parallel_calls=n_parse_threads)
    return dataset.prefetch(1)
```

```
train_set = quickdraw_dataset(train_files, shuffle_buffer_size=10000)
valid_set = quickdraw_dataset(eval_files[:5])
test_set = quickdraw_dataset(eval_files[5:])
```

```
for sketches, lengths, labels in train_set.take(1):
    print("sketches =", sketches)
    print("lengths =", lengths)
    print("labels =", labels)
```

```
sketches = tf.Tensor(
[[[-0.08627451  0.11764706  0.          ]
 [-0.01176471  0.16806725  0.          ]
 [ 0.02352941  0.07563025  0.          ]
 ...
 [ 0.          0.          0.          ]
 [ 0.          0.          0.          ]
 [ 0.          0.          0.          ]]]

[[-0.04705882 -0.06696428  0.          ]
 [-0.09019607 -0.07142857  0.          ]
 [-0.0862745  -0.04464286  0.          ]
 ...
 [ 0.          0.          0.          ]
 [ 0.          0.          0.          ]
 [ 0.          0.          0.          ]]]

[[ 0.          0.          1.          ]
 [ 0.          0.          0.          ]
 [ 0.00784314  0.11320752  0.          ]
 ...
 [ 0.11764708  0.01886791  0.          ]
 [-0.03529412  0.12264156  0.          ]
 [-0.19215688  0.33962262  1.          ]]]

...

[[-0.21276593 -0.01960784  0.          ]
 [-0.31382978  0.00784314  0.          ]
 [-0.37234044  0.13725491  0.          ]
 ...
 [ 0.          0.          0.          ]
 [ 0.          0.          0.          ]
 [ 0.          0.          0.          ]]]

[[ 0.          0.4677419  0.          ]
 [-0.01176471  0.15053767  0.          ]
 [ 0.16470589  0.05376345  0.          ]]
```



```

...
[ 0.      0.      0.      ]
[ 0.      0.      0.      ]
[ 0.      0.      0.      ]]

[[-0.04819274  0.01568627  0.      ]
 [-0.07228917 -0.01176471  0.      ]
 [-0.05622491 -0.03921568  0.      ]
...
[ 0.      0.      0.      ]
[ 0.      0.      0.      ]
[ 0.      0.      0.      ]]], shape=(32, 104, 3), dtype=float32)
lengths = tf.Tensor(
[ 29 48 104 34 29 35 28 40 95 26 23 41 47 17 37 47 12 13
 17 41 36 23 8 15 60 32 54 38 68 30 89 36], shape=(32,), dtype=int64)
labels = tf.Tensor(
[ 95 190 163 12 77 213 216 278 25 202 310 33 327 204 260 181 337 233
 299 186 61 157 274 150 7 34 47 319 213 292 312 282], shape=(32,), dtype=int64)

```

```

def draw_sketch(sketch, label=None):
    origin = np.array([[0., 0., 0.]])
    sketch = np.r_[origin, sketch]
    stroke_end_indices = np.argwhere(sketch[:, -1]==1.)[: , 0]
    coordinates = sketch[:, :2].cumsum(axis=0)
    strokes = np.split(coordinates, stroke_end_indices + 1)
    title = class_names[label.numpy()] if label is not None else "Try to guess"
    plt.title(title)
    plt.plot(coordinates[:, 0], -coordinates[:, 1], "y:")
    for stroke in strokes:
        plt.plot(stroke[:, 0], -stroke[:, 1], "-.")
    plt.axis("off")

def draw_sketches(sketches, lengths, labels):
    n_sketches = len(sketches)
    n_cols = 4
    n_rows = (n_sketches - 1) // n_cols + 1
    plt.figure(figsize=(n_cols * 3, n_rows * 3.5))
    for index, sketch, length, label in zip(range(n_sketches), sketches, lengths, labels):
        plt.subplot(n_rows, n_cols, index + 1)
        draw_sketch(sketch[:length], label)
    plt.show()

for sketches, lengths, labels in train_set.take(1):
    draw_sketches(sketches, lengths, labels)

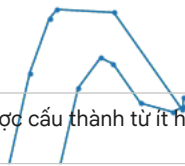
```



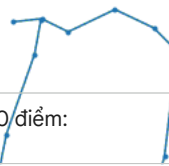
picture frame



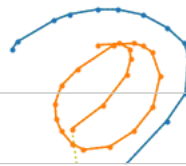
boomerang



square

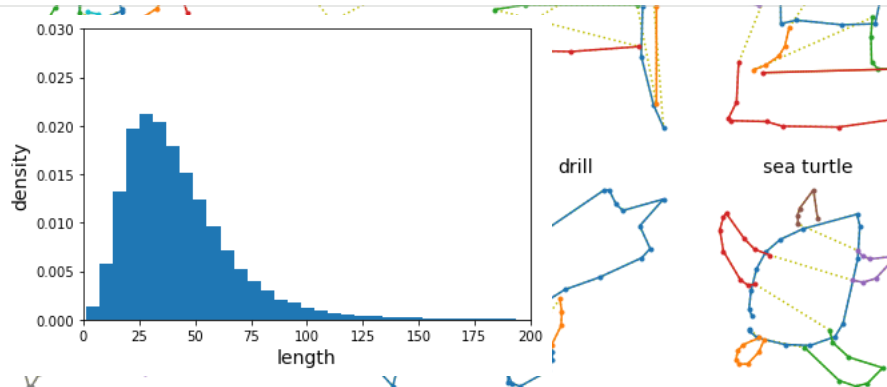


ear



Hầu hết các bản phác thảo được cấu thành từ ít hơn 100 điểm:

```
lengths = np.concatenate([lengths for _, lengths, _ in train_set.take(1000)])
plt.hist(lengths, bins=150, density=True)
plt.axis([0, 200, 0, 0.03])
plt.xlabel("length")
plt.ylabel("density")
plt.show()
```



```
def crop_long_sketches(dataset, max_length=100):
    return dataset.map(lambda inks, lengths, labels: (inks[:, :max_length], labels))

cropped_train_set = crop_long_sketches(train_set)
cropped_valid_set = crop_long_sketches(valid_set)
cropped_test_set = crop_long_sketches(test_set)
```

```
model = tf.keras.Sequential([
    tf.keras.layers.Conv1D(32, kernel_size=5, strides=2, activation="relu"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Conv1D(64, kernel_size=5, strides=2, activation="relu"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Conv1D(128, kernel_size=3, strides=2, activation="relu"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.LSTM(128, return_sequences=True),
    tf.keras.layers.LSTM(128),
    tf.keras.layers.Dense(len(class_names), activation="softmax")
])
optimizer = tf.keras.optimizers.SGD(learning_rate=1e-2, clipnorm=1.)
model.compile(loss="sparse_categorical_crossentropy",
              optimizer=optimizer,
              metrics=["accuracy", "sparse_top_k_categorical_accuracy"])
history = model.fit(cropped_train_set, epochs=2,
                  validation_data=cropped_valid_set)
```

```
Epoch 1/2
107813/107813 [=====] - 2048s 19ms/step - loss: 4.0817 - accuracy: 0.1705 - sparse_top_k_categorical_ac
Epoch 2/2
107813/107813 [=====] - 3975s 37ms/step - loss: 2.7176 - accuracy: 0.3771 - sparse_top_k_categorical_ac
```

```
y_test = np.concatenate([labels for _, _, labels in test_set])
y_probas = model.predict(test_set)
```

WARNING:tensorflow:5 out of the last 18 calls to <function Model.make\_predict\_function.<locals>.predict\_function at 0x7fd0e07f7a

```
np.mean(tf.keras.metrics.sparse_top_k_categorical_accuracy(y_test, y_probas))
```

0.60668993

parrot

church

bulldozer

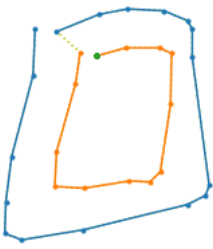
smiley face

```
n_new = 10
Y_probas = model.predict(sketches)
top_k = tf.nn.top_k(Y_probas, k=5)
for index in range(n_new):
```

```
plt.figure(figsize=(3, 3.5))
draw_sketch(sketches[index])
plt.show()
print("Top-5 predictions:".format(index + 1))
for k in range(5):
    class_name = class_names[top_k.indices[index, k]]
    proba = 100 * top_k.values[index, k]
    print("  {}. {} {:.3f}%".format(k + 1, class_name, proba))
print("Answer: {}".format(class_names[labels[index].numpy()])))
```



Try to guess

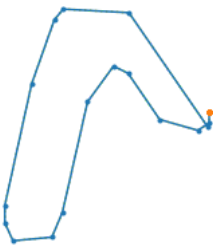


Top-5 predictions:

1. popsicle 13.105%
2. computer 7.943%
3. television 7.032%
4. laptop 6.640%
5. cell phone 5.520%

Answer: picture frame

Try to guess

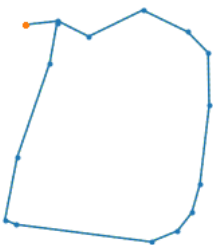


Top-5 predictions:

1. garden hose 15.217%
2. trumpet 10.083%
3. rifle 8.203%
4. spoon 5.367%
5. moustache 4.533%

Answer: boomerang

Try to guess

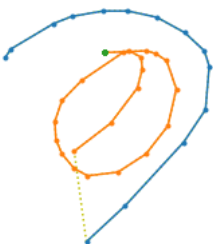


Top-5 predictions:

1. wine bottle 24.326%
2. hexagon 22.632%
3. octagon 13.903%
4. lipstick 2.759%
5. blackberry 2.112%

Answer: square

Try to guess

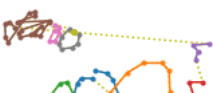


Top-5 predictions:

1. ear 62.866%
2. moon 17.284%
3. boomerang 3.729%
4. knee 2.912%
5. squiggle 2.257%

Answer: ear

Try to guess



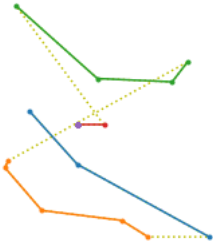


Top-5 predictions:

1. monkey 34.293%
2. mermaid 8.274%
3. blueberry 7.341%
4. camouflage 4.992%
5. bear 4.961%

Answer: monkey

Try to guess

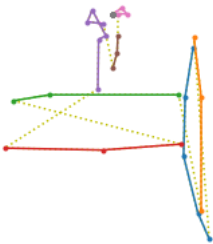


Top-5 predictions:

1. fork 8.643%
2. shovel 7.149%
3. syringe 6.684%
4. screwdriver 5.352%
5. stitches 4.247%

Answer: line

Try to guess



Top-5 predictions:

1. snowflake 22.972%
2. yoga 10.533%
3. matches 6.915%
4. candle 4.574%
5. syringe 3.947%

Answer: trumpet

Try to guess



Top-5 predictions:

1. shovel 15.070%
2. floor lamp 10.788%
3. screwdriver 10.516%
4. lipstick 9.559%
5. lantern 7.887%

Answer: anvil

Try to guess







```
[74, 70, 65, 46],
[74, 70, 65, 46],
[74, 70, 65, 46],
[74, 70, 65, 46],
[75, 69, 63, 48],
[75, 69, 63, 48],
[75, 67, 63, 48],
[75, 67, 63, 48],
[77, 65, 62, 50],
[77, 65, 62, 50],
[77, 65, 60, 50],
[77, 65, 60, 50],
[74, 67, 58, 55],
[74, 67, 58, 55],
[74, 67, 58, 53],
[74, 67, 58, 53],
[72, 67, 58, 51],
[72, 67, 58, 51],
[72, 67, 58, 51],
[72, 67, 58, 51],
[72, 65, 57, 53],
[72, 65, 57, 53],
[72, 65, 57, 53],
[72, 65, 57, 53],
[70, 65, 62, 46],
[70, 65, 62, 46],
```

Các nốt nhạc nằm trong khoảng từ 36 (C1) đến 81 (A5), cộng với 0 cho sự im lặng:

```
notes = set()
for chorales in (train_chorales, valid_chorales, test_chorales):
    for chorale in chorales:
        for chord in chorale:
            notes |= set(chord)

n_notes = len(notes)
min_note = min(notes - {0})
max_note = max(notes)

assert min_note == 36
assert max_note == 81
```

Hãy viết một vài hàm để nghe những bản hợp xướng này (tôi chỉ muốn viết một bộ tổng hợp âm thanh cho vui thôi):

```
from IPython.display import Audio

def notes_to_frequencies(notes):
    # Frequency doubles when you go up one octave; there are 12 semi-tones
    # per octave; Note A on octave 4 is 440 Hz, and it is note number 69.
    return 2 ** ((np.array(notes) - 69) / 12) * 440

def frequencies_to_samples(frequencies, tempo, sample_rate):
    note_duration = 60 / tempo # the tempo is measured in beats per minutes
    # To reduce click sound at every beat, we round the frequencies to try to
    # get the samples close to zero at the end of each note.
    frequencies = (note_duration * frequencies).round() / note_duration
    n_samples = int(note_duration * sample_rate)
    time = np.linspace(0, note_duration, n_samples)
    sine_waves = np.sin(2 * np.pi * frequencies.reshape(-1, 1) * time)
    # Removing all notes with frequencies ≤ 9 Hz (includes note 0 = silence)
    sine_waves *= (frequencies > 9.).reshape(-1, 1)
    return sine_waves.reshape(-1)

def chords_to_samples(chords, tempo, sample_rate):
    freqs = notes_to_frequencies(chords)
    freqs = np.r_[freqs, freqs[-1:]] # make last note a bit longer
    merged = np.mean([frequencies_to_samples(melody, tempo, sample_rate)
                      for melody in freqs.T], axis=0)
    n_fade_out_samples = sample_rate * 60 // tempo # fade out last note
    fade_out = np.linspace(1., 0., n_fade_out_samples)**2
    merged[-n_fade_out_samples:] *= fade_out
    return merged

def play_chords(chords, tempo=160, amplitude=0.1, sample_rate=44100, filepath=None):
    samples = amplitude * chords_to_samples(chords, tempo, sample_rate)
    if filepath:
        from scipy.io import wavfile
```